Thesis of the end-of-study project

To obtain the State Engineering Diploma

Major: Computer Engineering

STM32Cube Competitor Benchmark

Authored by :

Fares Dkhili

Defended on September 2026, before the jury composed of :

Mrs. Anissa Omrane	FST	- President
Mrs. Sana Younes	FST	- Examiner
Mr. Aymen Zemrani	FST	- Academic supervisor
Mrs. Sirine Eljazi	STMicroelectronics	- Professional supervisor

Class of : 2025/2026

Dedicated to

To my dear parents, whose unconditional love, patience, and countless sacrifices have carried me to this moment. Everything I have achieved is a reflection of the values you taught me.

To my family and my friends, who never stopped believing in me and who lifted me through every doubt and every long night. This work is as much yours as it is mine.

Fares Dkhili.

Acknowledgments

This project marks the end of my engineering studies, and it could not have come together without the people who guided and supported me along the way.

I am deeply grateful to Ms. Sirine Eljazi, my supervisor at STMicroelectronics, for her trust, her availability, and the technical insight she shared throughout this work. Her guidance shaped both the project and the engineer I have become. I also thank the entire STMicroelectronics team for the warm welcome and for making my time within the company a genuine learning experience.

I would like to express my sincere thanks to Mr. Aymen Zemrani, my academic supervisor at the Faculty of Sciences of Tunis (FST), for his rigorous follow-up, his sound advice, and his constant encouragement from the first day to the last.

My thanks also go to the members of the jury: to Ms. Anissa Omrane for the honor of chairing the jury, and to Ms. Sana Younes for accepting to examine and evaluate this work. Their time and attention are sincerely appreciated.

I extend my gratitude to the teaching staff of the Faculty of Sciences of Tunis at the University of Tunis El Manar, whose dedication over these years laid the foundations on which this project was built.

Finally, my heartfelt thanks to my family and friends, whose unwavering support and patience accompanied me through every stage of this journey.

Résumé

Ce rapport présente les travaux réalisés dans le cadre d'un projet de fin d'études au sein de STMicroelectronics Tunis, portant sur le benchmarking de l'écosystème STM32Cube face aux écosystèmes concurrents (GigaDevice GD32, NXP, Texas Instruments, Renesas). L'objectif est de fournir à l'équipe de marketing technique des données comparatives objectives, reproductibles et interprétables sur l'empreinte mémoire (flash et RAM), les performances d'exécution et la maturité des piles logicielles embarquées.

Nous avons conçu une méthodologie de benchmarking structurée reposant sur des scénarios identiques exécutés sur les deux plateformes comparées, avec le même niveau d'optimisation, la même fréquence d'horloge et le même comportement observable. Deux outils complémentaires ont été développés en parallèle pour rendre cette méthodologie scalable et reproductible : *MCU Workflow Studio*, une application d'analyse d'empreinte firmware inter-vendeurs qui automatise la comparaison de fichiers linker map, la classification par couche architecturale et l'export des résultats ; et le *MCU Benchmark Copilot Agent*, un système d'agents IA autonome personnalisé intégré à VS Code qui pilote des campagnes de benchmarking de bout en bout — de la résolution de scénario et l'acquisition de firmware jusqu'à l'automatisation de build IAR, le débogage guidé par les preuves, l'extraction de mesures et la vérification d'équivalence sémantique — ne demandant l'approbation de l'utilisateur qu'avant la programmation du matériel physique.

Les résultats obtenus sur la comparaison ST vs GD32 — couvrant la pile USB device (scénarios HID et CDC) et l'intégration FreeRTOS (scénarios basique, coopératif et préemptif) — révèlent un compromis architectural clair : STM32 consomme davantage de ROM (35–88 %) en raison de sa couche HAL modulaire, mais utilise significativement moins de RAM (57–65 % pour l'USB) et offre des performances d'initialisation supérieures (61,7 % plus rapide). Des propositions d'amélioration ciblées sont formulées, notamment l'optimisation du module HAL RCC et la sélection fine des fonctionnalités USB à la compilation.

Mots clés : STM32Cube, benchmarking, empreinte mémoire, microcontrôleur, analyse de fichier

linker map, FreeRTOS, USB, GD32, portage inter-vendeurs, agent IA.

Summary

This report presents work carried out during an end-of-studies internship at STMicroelectronics Tunis, focused on benchmarking the STM32Cube ecosystem against competing vendor ecosystems (GigaDevice GD32, NXP, Texas Instruments, Renesas). The goal is to provide the technical marketing team with objective, reproducible, and interpretable comparative data on firmware memory footprint (flash and RAM), execution performance, and middleware maturity.

We designed a structured benchmarking methodology based on identical scenarios executed on both compared platforms under the same optimisation level, clock frequency, and observable behaviour. Two complementary tools were developed in parallel to make this methodology scalable and reproducible: *MCU Workflow Studio*, a cross-vendor firmware footprint analysis application that automates linker-map comparison, architectural-layer classification, and result export; and the *MCU Benchmark Copilot Agent*, a custom autonomous AI agent system integrated into VS Code that drives end-to-end benchmark campaigns — from scenario resolution and firmware acquisition through IAR build automation, evidence-driven debugging, measurement extraction, and semantic-equivalence verification — asking the user only before programming physical hardware.

Results from the ST-versus-GD32 comparison — covering the USB device stack (HID and CDC scenarios) and FreeRTOS integration (basic, cooperative, and preemptive scheduling scenarios) — reveal a clear architectural trade-off: STM32 consumes more ROM (35–88 %) due to its modular HAL layer, but uses significantly less RAM (57–65 % for USB) and delivers superior initialisation performance (61.7 % faster). Targeted enhancement proposals are formulated, notably optimising the HAL RCC module and enabling finer compile-time USB feature selection.

Key Words: STM32Cube, benchmarking, memory footprint, microcontroller, linker-map analysis, FreeRTOS, USB, GD32, cross-vendor porting, AI agent.

ملخص

يقدّم هذا التقرير الأعمال المنجزة في إطار مشروع نهاية الدراسات بشركة إس تي مايكروإلكترونيكس بتونس، والتي تتمحور حول قياس أداء المنظومة البرمجية صةپ٢٣ثب مقارنةً بالمنظومات المنافسة (جگضشڤ جض٢٣، خع، ةشس اينسترمنتس، غنسس). يهدف المشروع إلى تزويد فريق التسويق التقني ببيانات مقارنة موضوعية وقابلة للتكرار حول البصمة الذاكرية (فلاش وغاپ) وأداء التنفيذ ونضج الطبقات البرمجية الوسيطة.

صمّمنا منهجية قياس أداء مهيكلة تعتمد على سيناريوهات متطابقة تُنفَذْ على كلتا المنصتين المقارنتين بنفس مستوى التحسين وتردد الساعة والسلوك الملاحظ. طوّرنا بالتوازي أداتين متكاملتين: تطبيق MCU Workflow Studio لتحليل البصمة الذاكرية عبر مقارنة ملفات الربط وتصنيف الرموز حسب الطبقة المعمارية، ونظام MCU Benchmark Copilot Agent المستقل المدمج في بيئة VS Code الذي يقود حملات القياس من البداية إلى النهاية من حل السيناريو واكتساب البرامج الثابتة إلى أتمتة البناء والتصحيح واستخراج القياسات والتحقق من التكافؤ الدلالي دون تدخل المستخدم إلا عند برمجة العتاد الفعلي.

كشفت نتائج المقارنة بين صة وجض٢٣ على مستوى مكدس اوصـ (سيناريوهات حيض وثضث) وتكامل ذرلغةوص (سيناريوهات المعالجة الأساسية والتعاونية والاستباقية) عن مقايضة معمارية واضحة: يستهلك صةپ٢٣ ذاكرة غوپ أكثر (٨٨--٣٥ %) بسبب طبقة حای المعيارية، لكنه يستخدم ذاكرة غاپ أقل بكثير (٦٥--٥٧ % لـوصـ) ويوفر أداء تهيئة أسرع بنسبة ٦١٫٧ %. قُدّمت مقترحات تحسين مستهدفة، أبرزها تحسين وحدة حای غثث وتمكين الاختيار الدقيق لميزات اوصـ عند التجميع.

الكلمات المفتاحية:

STM32Cube ، قياس الأداء، البصمة الذاكرية، متحكم دقيق، تحليل ملف الربط، FreeRTOS ، USB ، GD32 ، النقل بين المصنّعين، وكيل ذكاء اصطناعي.

List of Figures

List of Tables

Listings

Contents

Chapter 1

General Description of the Project

Introduction

This chapter introduces the host company and the context of the project. The existing problems are detailed to define the specifications and clarify the requested work. We then present the theoretical foundations relevant to the project and explain the adopted work methodology.

1.1 Host company

This section provides an overview of STMicroelectronics, its history, the ST Tunis site, and the technically oriented marketing and application support team in which this project was carried out.

1.1.1 Presentation

STMicroelectronics is a global semiconductor company dedicated to enhancing people's lives through innovative electronic solutions [1]. With one of the most extensive product portfolios in the industry, ST serves over 200,000 customers worldwide, achieving net revenues of $13.27 billion in 2024 [2].

The company was established in June 1987 as SGS-THOMSON Microelectronics, following the merger of SGS Microelettronica (Italy) and Thomson Semiconductors (France). The name "STMicroelectronics" was officially adopted in May 1998. Figure 1.1 shows the company's official logo.

Figure 1.1: STMicroelectronics logo.

With more than 80 sales and marketing offices and 14 primary manufacturing locations across the globe, the corporation employs over 50,000 people [1].

1.1.2 STMicroelectronics vision and markets

STMicroelectronics has prioritised technological innovation in its strategy for over 25 years. It makes market-driven investments in technology development with the goal of commercialising cutting-edge innovations that create immediate value for its customers [3]. Today, with a robust pipeline, ST delivers products that are smaller, faster, more energy-efficient, and equipped with new features. Driven by a global lean manufacturing culture, ST addresses three main end markets: automotive, industrial, and personal electronics — and offers engagement models ranging from application-specific embedded systems to application-specific standard products.

1.1.3 STMicroelectronics Tunis

The ST Tunis site, located in Technopark El Ghazela, began operations in 2001 with nine engineers. Today it has expanded to more than 200 employees working across design, tools, applications, quality, and support functions [1]. The facility participates in every stage from the initial design of new circuits to the final verification of their functionality before mass production.

The Microcontroller Division (MCD) is dedicated to the design of microcontrollers, demonstration platforms, and solutions for consumer use. It is responsible for the development of drivers and embedded software — specifically the validation and development of libraries and firmware that drive ST microcontrollers. Figure 1.2 shows the organisational structure of the MCD division at the Tunis site; the Maintenance team, highlighted, is where this project was carried out.

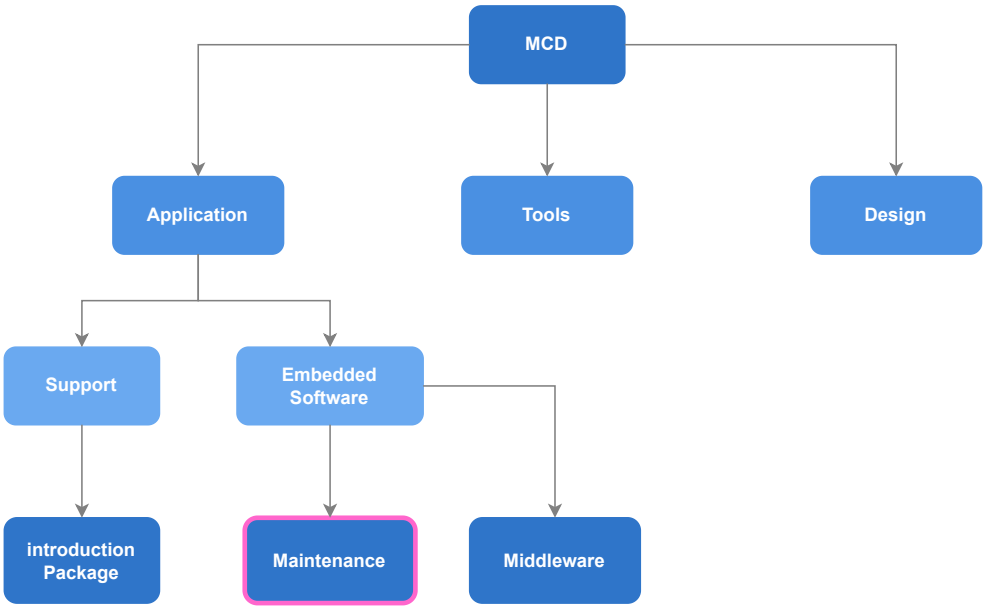

Figure 1.2: Organisational structure of the Microcontroller Division (MCD) at ST Tunis. The Maintenance team (highlighted) is the host team of this project.

This project was carried out in the **Technical Marketing and Application Support** team under MCD. This team includes two sub-teams:

- **The GitHub team**: supports customers via GitHub, publishes updates of HAL drivers, and maintains example projects to facilitate application development based on STM32 microcontrollers.
- **The Maintenance and Validation team**: ensures customer satisfaction through bug maintenance and functional validation of manufactured microcontrollers and their associated HAL versions. This team represents the junction linking software and hardware design processes to implementation and industrialisation.

1.2 Project presentation

1.2.1 Project framework

As part of our training in Computer Engineering at the Faculty of Sciences of Tunis (FST), University of Tunis El Manar (UTM), we conclude our education with an end-of-studies project (PFE). In this context, our project — entitled *STM32Cube Competitor Benchmark* — took place within STMicroelectronics Tunis over a period of six months (February–September 2026).

1.2.2 Project context

The embedded microcontroller market is highly competitive. When an engineer evaluates an MCU for a new design, the choice between STMicroelectronics and competing vendors (GigaDevice GD32, NXP, Texas Instruments, Renesas) often depends on measurable differences: firmware footprint, execution speed, middleware maturity, and development-tool productivity. The technical marketing team at ST needs objective, reproducible data that quantifies where the STM32Cube ecosystem outperforms its competitors, where gaps exist, and what concrete enhancements would close those gaps.

At the start of this project, no systematic benchmarking methodology existed within the team. Cross-vendor comparisons were ad-hoc, performed manually on a case-by-case basis, and difficult to reproduce or communicate convincingly to stakeholders.

1.2.3 Problematic

Producing actionable competitive intelligence for the marketing team requires answering precise questions: for a given software stack (e.g. USB device, FreeRTOS integration), running identical scenarios on ST and on a competitor, how do the two platforms compare in flash and RAM footprint, execution performance, and middleware completeness? And where ST lags, what specific enhancements — at the HAL, middleware, or configuration level — would close the gap?

Answering these questions rigorously raises several challenges:

1. **Scenario design**: each benchmark must exercise the same functionality on both platforms under identical, well-defined conditions so that the comparison is fair and the results are meaningful.

2. **Porting fidelity**: reproducing an STM32 reference scenario on a competitor requires preserving exact runtime semantics — task structure, timing, synchronisation, reporting format — to avoid comparing apples to oranges.

3. **Result extraction and interpretation**: raw numbers (kilobytes, cycles, milliseconds) must be attributed to comparable architectural layers and interpreted in terms of engineering trade-offs, not merely listed.

4. **Scale and reproducibility**: the team needs to cover multiple vendors and multiple stacks with consistent methodology, so that results are comparable across campaigns and over time.

1.2.4 Proposed solution

Our solution is a structured benchmarking methodology centred on three deliverables:

1. **Benchmark scenarios**: for each software stack under evaluation (USB device stack, FreeRTOS, etc.), we design a set of concrete, reproducible scenarios that both the STM32 reference and the competing platform must run identically. Each scenario exercises a specific aspect of the stack (e.g. HID, CDC, and MSC for USB device).

2. **Comparative results and interpretation**: for each scenario, we extract per-layer footprint figures, execution metrics, and qualitative observations, then interpret them to show where ST is stronger, where the competitor leads, and why.

3. **Enhancement proposals**: where the data reveals a gap unfavourable to ST, we identify the root cause and propose targeted improvements — whether at the HAL driver level, in middleware configuration, or in linker/startup overhead.

To support this workflow at scale and ensure reproducibility, we developed two supportive tools in parallel:

- **MCU Workflow Studio** — a cross-vendor firmware footprint analysis application that automates linker-map comparison, layer classification, and result export (detailed in Chapter 2).

- **MCU Benchmark Copilot Agent** — a custom AI agent system that accelerates benchmark creation, porting, and debugging while enforcing fairness and semantic equivalence (detailed in Chapter 3).

These tools are parallel engineering contributions that enhance the benchmarking effort; they are not the benchmarks themselves. The core deliverable remains the benchmark results and the enhancement proposals they yield.

1.2.5 Mission and objectives

The mission of this internship is to provide the STMicroelectronics technical marketing team with rigorous, reproducible, and interpretable competitive benchmark data. Specifically, the project aims to:

1. Design benchmark scenarios for selected software stacks, ensuring fairness and semantic equivalence between ST and each competitor.

2. Execute the benchmarks on representative hardware, extracting flash/RAM footprint, execution performance, and qualitative observations per scenario.

3. Compare and interpret the results layer by layer, highlighting where ST excels and where gaps exist.

4. Formulate concrete enhancement proposals for STM32Cube where the data reveals room for improvement.

5. Develop supportive tools (footprint analyser, benchmark agent) to make the methodology reproducible and scalable beyond this project.

1.3 Theoretical foundations

This section presents the background concepts essential for understanding the technical aspects of the project: the STM32 microcontroller ecosystem, firmware memory footprint, and the linker-map artefact that our tools consume.

1.3.1 STM32 microcontrollers

1.3.1.1 Overview

A microcontroller unit (MCU) is an integrated circuit that combines a processor core, memory (flash and RAM), and peripherals on a single chip. It is designed for embedded applications where cost, power, and real-time determinism are critical constraints.

1.3.1.2 The STM32 family

The STM32 family from STMicroelectronics is a series of 32-bit microcontrollers based on ARM Cortex-M processor cores, produced since 2007 [4]. The family spans four macro groups — high-performance, mainstream, ultra-low-power, and wireless — with more than ten product sub-families. Each sub-family targets a different balance of processing power, memory, peripherals, and energy efficiency.

The ARM Cortex-M profile is optimised for embedded applications requiring low cost and low power consumption [5]. Cortex-M cores range from the M0 (simplest, lowest gate count) to the M7 and M33 (highest performance, with DSP and optional floating-point units). STM32 devices span this entire range.

1.3.1.3 The STM32Cube ecosystem

STMicroelectronics provides a comprehensive software ecosystem around its STM32 hardware [6]:

- **STM32CubeMX**: a graphical configuration tool that generates initialisation code for peripherals, clocks, and middleware.
- **STM32CubeIDE**: an integrated development environment based on Eclipse, with build, flash, and debug support.
- **STM32Cube firmware packages**: per-family packages containing the HAL and low-level (LL) drivers, middleware (FreeRTOS, USB, FatFS, LwIP, mbedTLS), board support packages (BSP), and example projects.
- **STM32CubeProgrammer**: a flashing and option-byte configuration utility.
- **STM32CubeMonitor**: a real-time variable monitoring tool.

It is this ecosystem — not merely the silicon — that we benchmark against competitors. When we compare "ST versus GD32," we compare the full software stack each vendor provides for a given use case, because that is what an engineer evaluates when choosing a platform.

1.3.2 Firmware memory footprint

The memory footprint of firmware is the amount of non-volatile (flash) and volatile (RAM) memory it occupies once compiled and linked. Flash stores read-only code and constant data; RAM stores read-write data (global and static variables, plus stack and heap at run time).

Footprint is a first-class metric in MCU benchmarking for several reasons. Flash and RAM are fixed, on-chip resources; every kilobyte consumed by the vendor's software stack is a kilobyte unavailable for application logic. A smaller footprint enables the use of a cheaper device with less memory, directly affecting bill-of-materials cost. It also leaves headroom for future features and reduces power consumption in flash-retention scenarios.

Comparing footprint across vendors is non-trivial because each vendor structures, names, and layers its code differently. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, `GPIO_PinInit` on NXP, and `R_IOPORT_Open` on Renesas. A fair comparison must map these correspondences and attribute costs to comparable architectural layers (application, middleware, low-level driver, system runtime).

1.3.3 Linker maps

The linker map is a text file produced by the linker at the end of the build process. For the IAR Embedded Workbench for ARM (EWARM) toolchain [15], this file lists every module and function that was linked into the final image, together with its code size (read-only), constant-data size (read-only), and variable-data size (read-write). It also records the memory sections, placement addresses, and total resource usage.

The linker map is the single input to our footprint analysis tool. By parsing it, we can reconstruct the complete static memory budget of a firmware build without instrumenting or executing the code. This makes the measurement perfectly reproducible: the same source code, compiler version, and optimisation level always produce the same map.

1.4 Work methodology

We adopted a "divide and conquer" approach [7], decomposing the project into well-scoped tasks with clear deliverables and deadlines:

1. **Task 1 — Literature and ecosystem study**: research the STM32Cube ecosystem, competing vendor ecosystems (GD32, NXP, TI, Renesas), linker-map formats, and existing benchmarking practices.

2. **Task 2 — Benchmark scenario design**: define the software stacks to benchmark, select representative boards for ST and each competitor, and design fair, reproducible scenarios with identical observable behaviour on both sides.

3. **Task 3 — Benchmark execution and porting**: build and validate each scenario on the STM32 reference, port it to the competitor platform preserving semantic equivalence, compile both under IAR EWARM, and extract results.

4. **Task 4 — Result comparison and interpretation**: compare per-layer footprint, execution metrics, and qualitative observations; identify where ST excels and where gaps exist; formulate enhancement proposals.

5. **Task 5 — Supportive tool development** (parallel): design and implement MCU Workflow Studio (footprint analysis) and the MCU Benchmark Copilot Agent (benchmark creation and porting assistant) to make the methodology scalable and reproducible.

6. **Task 6 — Documentation and report**: document the methodology, tools, and results in this report.

[PLACEHOLDER — Gantt chart]

Capture or export the project Gantt chart showing Tasks 1–6 across February–September 2026.

Figure 1.3: Gantt chart of the project schedule (February–September 2026).

Conclusion

In this chapter, we introduced STMicroelectronics and its Tunis site, presented the competitive context that motivates cross-vendor benchmarking, and defined the core mission: design benchmark scenarios, execute them on ST and competitor platforms, compare and interpret the results for the marketing team, and propose targeted enhancements where ST can improve. Two supportive tools developed in parallel make this methodology reproducible and scalable. The theoretical foundations — the STM32Cube ecosystem, firmware footprint, and linker maps — provide the vocabulary for the chapters that follow. Chapter 2 and Chapter 3 present the supportive tools, while subsequent chapters present the benchmark results themselves.

Chapter 2

A Cross-Vendor Firmware Footprint Analysis Tool

Introduction

Comparing STMicroelectronics against a competing vendor is, at bottom, a measurement problem: the same workload is built for both platforms and the resulting binaries are weighed against one another. One of the most revealing measurements is the *memory footprint* of the firmware — how much flash and RAM a given software stack consumes once compiled and linked. Footprint is easy to define but tedious to compare fairly across ecosystems, because two vendors rarely name, split, or organise their code the same way. To make this comparison reproducible rather than artisanal, we designed and built a dedicated desktop and command-line application, *MCU Workflow Studio*, that ingests the linker output of two builds and produces a layer-aware, explainable footprint comparison between an STM32 reference and a competitor.

This chapter documents that tool as an engineering contribution in its own right. We first state the problem it solves and the requirements we set for it, then describe its architecture and its end-to-end pipeline, the implementation choices behind its deterministic core and its optional language-model assistance, how it plugs into the wider benchmarking effort, and finally its concrete value and its current limitations.

2.1 The cross-vendor footprint problem

The flash and RAM budget of a microcontroller unit (MCU) is a hard constraint. Every kilobyte of code competes with application logic for a fixed amount of on-chip memory, and it ripples into device cost, power, and the headroom left for future features. When we benchmark a software stack — a Universal Serial Bus (USB) device stack, a real-time operating system integration, and so on — the footprint each vendor charges for the same functionality is therefore a first-class metric.

Measuring it fairly is the difficult part. A modern build links application code against a vendor software development kit (SDK) whose drivers, middleware, and startup code are structured to that vendor's own conventions. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, as `GPIO_PinInit` on one competitor, and as `fsl_gpio` routines on another; equivalent middleware sits in different modules, and the boundary between "driver" and "system runtime" is drawn differently in each toolchain. Doing the comparison by hand means reading two linker maps side by side, guessing which symbol on one side corresponds to which on the other, and summing kilobytes into comparable buckets — slow, subjective, and impossible to reproduce exactly two weeks later.

We set the tool a small number of firm requirements. It had to be **deterministic and reproducible**, so that the same inputs always yield the same verdict. It had to be **layer-aware**, grouping symbols into architectural layers so that "ST spends more in the hardware abstraction layer (HAL)" can be stated precisely. Its symbol matching had to be **explainable**, attaching a confidence and a reason to every correspondence rather than hiding behind a black box. It had to treat **ST as the reference** against which each competitor is measured. And it had to **export** its results in formats the rest of the report can consume directly.

2.2 Design and architecture

The tool is organised as a thin set of user interfaces wrapped around a deterministic comparison core, with an SQLite-backed knowledge base alongside and an optional, off-by-default block of language-model assistance. Figure 2.1 shows the arrangement. Data flows top to bottom: a build's linker map enters through the ingestion layer, is classified into architectural layers, is compared symbol by symbol in the core, and leaves as a set of exported artefacts.

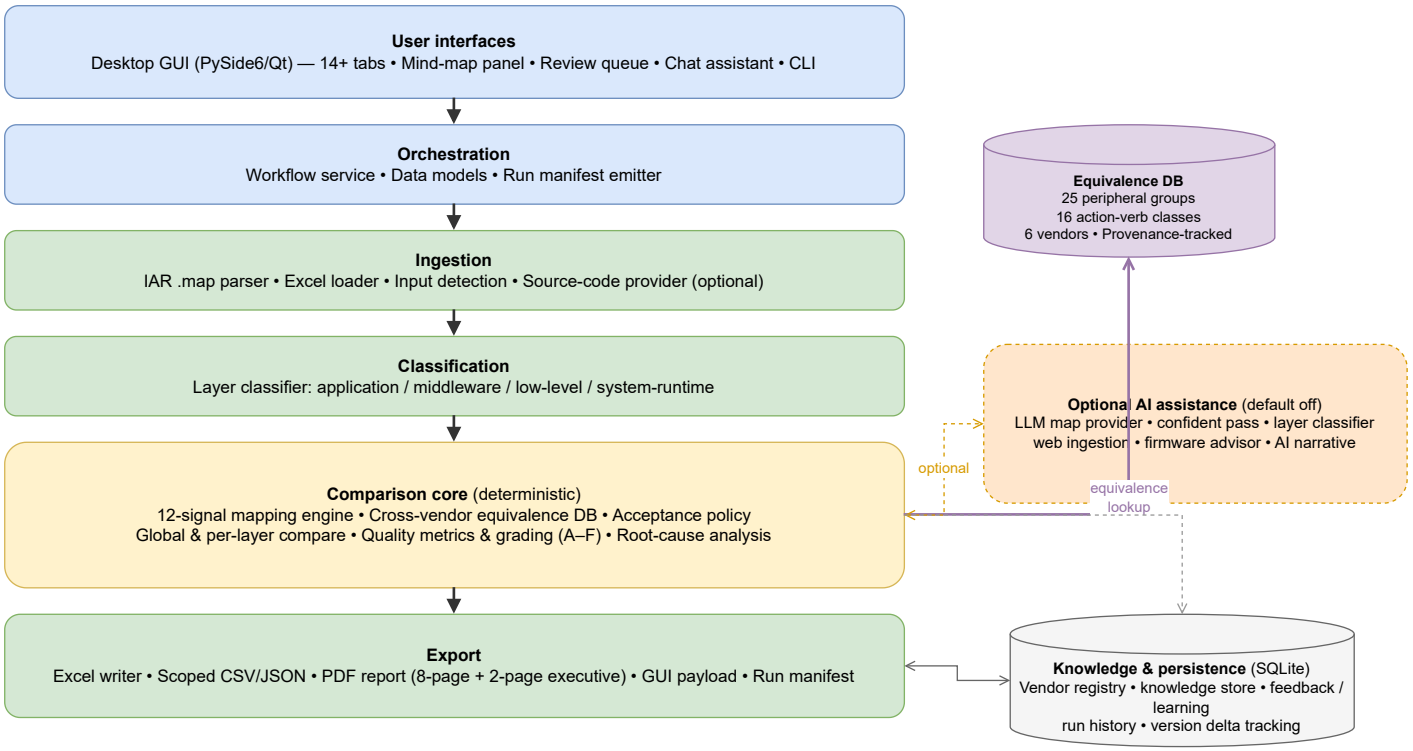

Figure 2.1: Layered architecture of the footprint-analysis tool. A deterministic comparison core (ingestion, classification, mapping, comparison, export) is wrapped by the user interfaces and backed by an SQLite knowledge base; an optional, off-by-default AI-assistance column refines the mapping without ever sitting on the critical path.

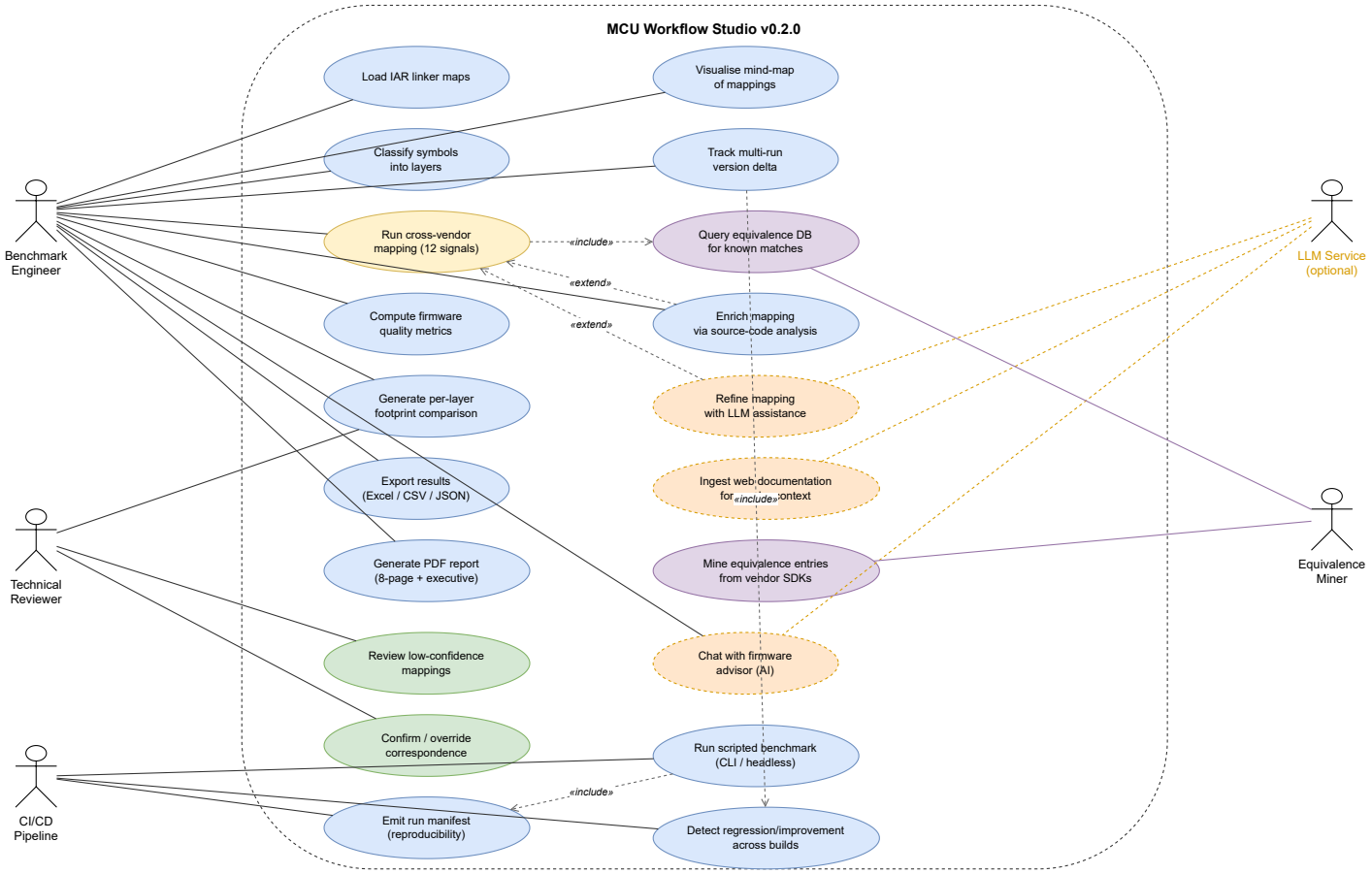

Figure 2.2: UML use-case diagram of MCU Workflow Studio v0.2.0. Five actors interact with twenty use cases grouped by concern: the benchmark engineer drives the core analysis, the technical reviewer validates low-confidence mappings, the CI/CD pipeline runs scripted headless benchmarks, the LLM service (optional, dashed) enriches the mapping, and the equivalence miner maintains the cross-vendor knowledge base.

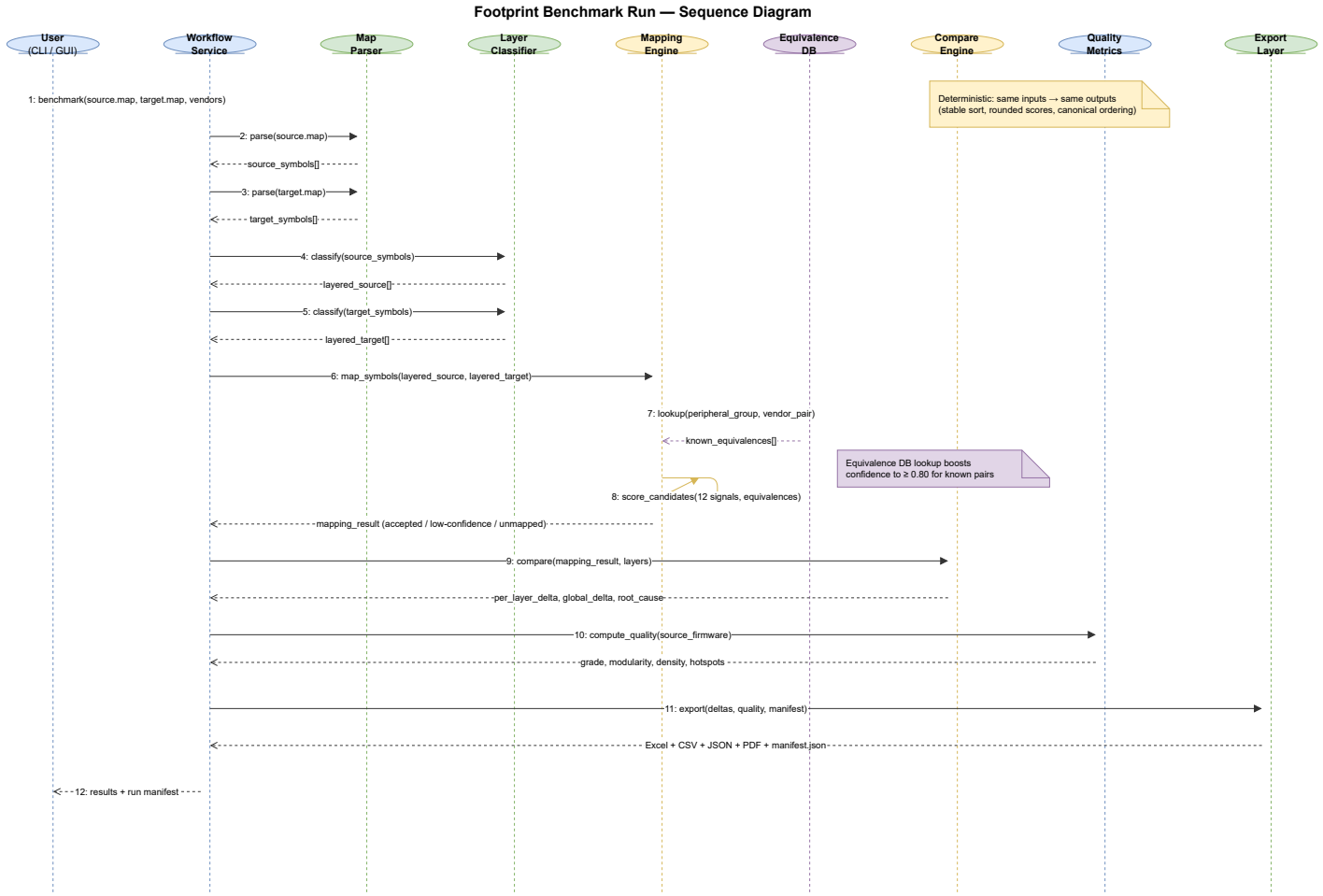

Figure 2.3: UML sequence diagram of a complete footprint benchmark run. The workflow service orchestrates twelve steps: parsing both linker maps, classifying symbols into layers, querying the equivalence database, scoring candidates with twelve weighted signals, computing per-layer deltas, evaluating firmware quality metrics, and emitting the run manifest alongside all export artefacts.

Two interfaces sit on top of the same engine: a desktop graphical user interface (GUI) built with PySide6 (the Qt binding for Python) [9], which provides over fourteen analysis tabs — including interactive charts, a mapping mind-map visualisation panel, a review queue, and a project-scoped chat assistant — and a command-line interface for scripted and reproducible runs. Both call into a single *workflow service* that owns the data models and drives every stage. Persistence is handled by SQLite [13]: a vendor registry of aliases and naming prefixes, a knowledge store of confirmed correspondences, an optional feedback store, and a history of past runs. The block of language-model features on the right is genuinely optional — the deterministic path produces a complete result without it — and we return to it in Section 2.4.3. Table 2.1 lists what the tool consumes and what it produces.

Table 2.1: Principal inputs and outputs of the tool.

Inputs	**Description**
IAR EWARM `.map` (×2)	Linker maps of the source (STM32) and target (competitor) builds; the only required inputs.
Prior workbook (`.xlsx`)	Optional earlier results re-used as a baseline.
Vendor identifiers	Source and target vendor names (default `stm32`/`nxp`) that select the alias and prefix tables.
Outputs	**Description**
`benchmark_results.xlsx`	Per-layer and global flash/RAM comparison with a verdict.
Scoped CSV/JSON/XLSX	Filtered exports for downstream processing.
Review queue (JSON/CSV)	Low-confidence mappings prioritised for manual confirmation.
Mapping diagnostics (JSON)	Per-pair signal breakdown and acceptance decisions.
GUI payload (JSON)	Pre-rendered data for the desktop interface.
PDF report (optional)	Eight-page narrative summary with charts, quality grade, and enhancement roadmap.
Run manifest (JSON)	Tool version, input checksums, equivalence DB hash, and deterministic mode for reproducibility.

2.3 The benchmarking pipeline

A benchmark run is a fixed sequence of stages, sketched in Figure 2.4. The solid path is purely deterministic; the dashed stages are the optional language-model steps, which are disabled by default and can be switched on without changing anything else. After validating its inputs, the tool parses both linker maps into a common in-memory model — build metadata, an overall footprint, and the list of modules and functions with their code and data sizes. Each symbol is then classified into an architectural layer, and the vendor knowledge (aliases, prefixes, prior matches) is prepared.

The heart of the run is the mapping stage, where each STM32 symbol is matched to its closest competitor counterpart by combining several similarity signals into a single, explainable score. The accepted matches are then aggregated: the tool computes the flash and RAM totals per layer and globally for both sides, compares them, and emits a verdict together with recommendations and a root-cause breakdown of where the difference comes from. Finally it persists the quality of the pair and

composes its exports; the optional advisor, PDF report, and run-history entry are produced only if requested.

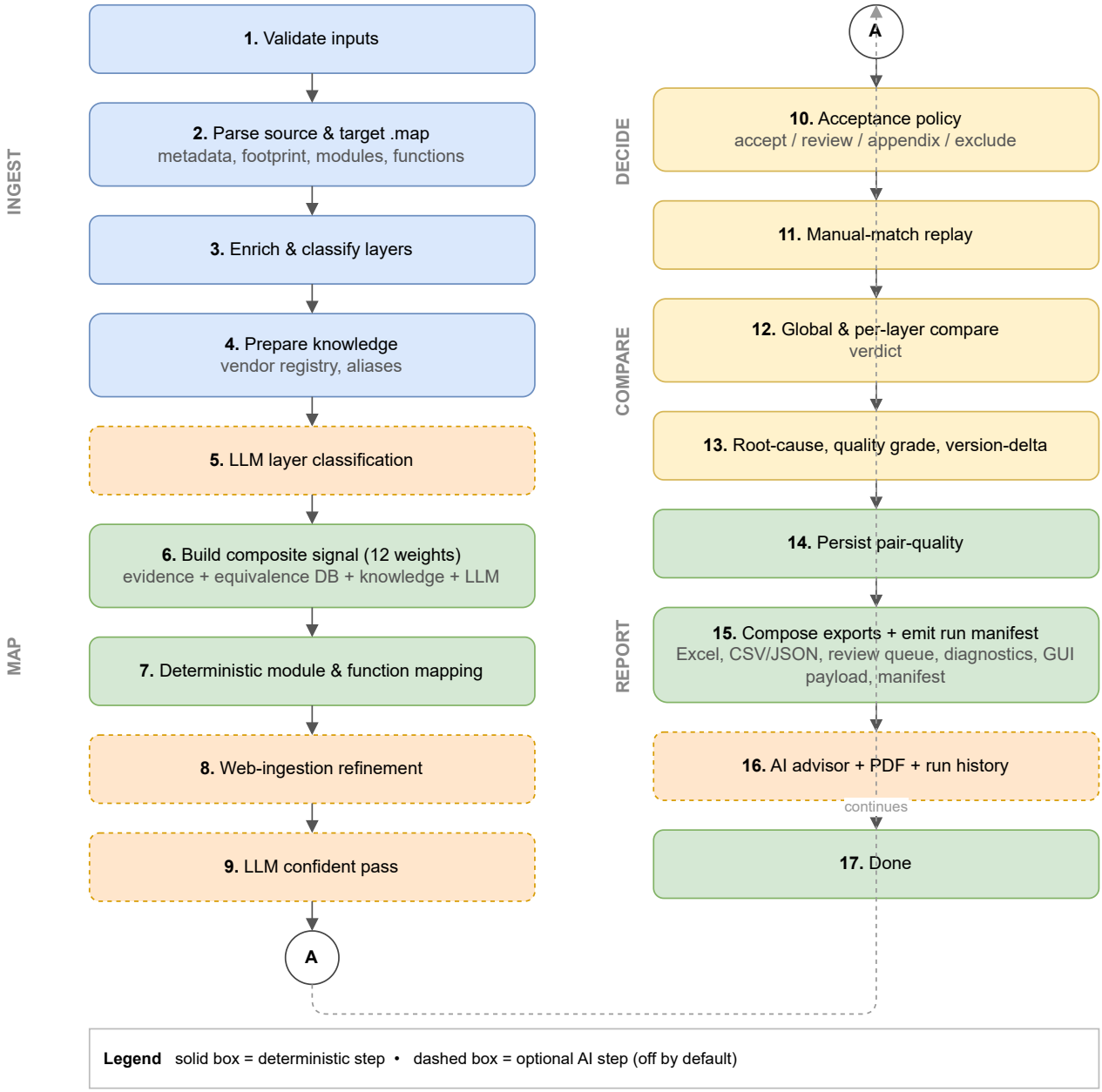

Figure 2.4: End-to-end benchmark pipeline, from linker-map ingestion to report generation. Dashed stages are optional AI-assisted steps that are disabled by default; the solid deterministic path runs end to end on its own.

A second, lighter mode produces a single-map footprint summary — validate, parse, classify, and write a footprint workbook — which is useful when only one build is available or to sanity-check a map before a full comparison.

2.4 Implementation

The tool is written entirely in Python ($\geq$ 3.11) [8], with a small, deliberately conventional dependency set summarised in Table 2.2. We favoured the standard library wherever it was sufficient: all persistence uses the built-in SQLite engine [13], and every network call to a language model goes through the standard `urllib` module rather than a vendor SDK, which keeps the executable small and the dependency surface auditable. The whole application can be packaged into a stand-alone Windows executable with PyInstaller [14].

Table 2.2: Implementation stack and the role of each external dependency.

Concern	**Technology**	**Role in the tool**
Language	Python $\geq$ 3.11 [8]	Entire code base (version 0.2.0).
Desktop GUI	PySide6 / Qt [9]	14+ tab windows, mind-map panel, chat assistant.
Tabular export	pandas [10]	Scoped CSV/JSON/Excel export.
Excel I/O	openpyxl [11]	Reads prior workbooks, writes results.
Charts & PDF	matplotlib [12]	Figures in the 8-page PDF report.
Persistence	SQLite [13]	Knowledge base, equivalence store, run history.
Packaging	PyInstaller [14]	Stand-alone Windows executable.

2.4.1 Parsing linker maps and classifying layers

The single required input is the linker map emitted by the IAR Embedded Workbench for ARM (EWARM) toolchain [15]. The parser reads this map and reconstructs, for each module and function, the read-only code and data it contributes and the read-write data it reserves. From these we derive the two footprint figures the benchmark cares about: flash occupancy as the sum of read-only code and read-only data, and RAM occupancy as the read-write data. This is a *static* measurement taken from the linked image; it captures what the firmware costs to store, not its dynamic stack or heap behaviour at run time, and we are careful to present it as such.

Each symbol is then assigned to one of four architectural layers — application, middleware, low-level driver, or system runtime — by a deterministic classifier driven by module names, known vendor prefixes, and section attributes. This layering is what lets the final report say not merely that one firmware is larger, but *where* it is larger, which is far more useful when the goal is to understand a vendor's engineering trade-offs. Figure 2.5 details the decision flow.

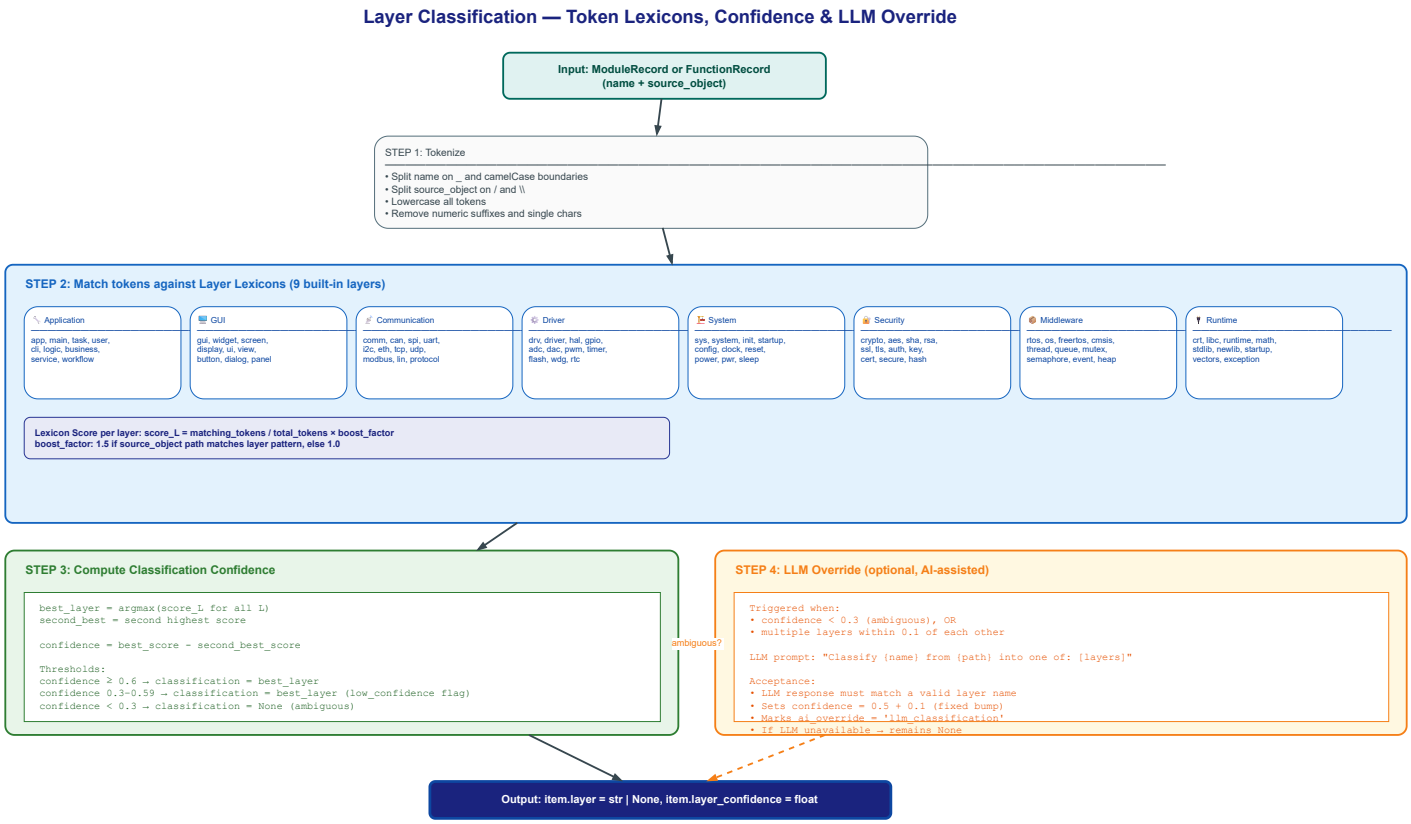

Figure 2.5: Layer-classification decision flow. Each symbol's tokens are matched against priority-ordered lexicons (system, middleware, low-level, application); the first match determines the layer with an associated confidence.

2.4.2 The multi-signal mapping engine

Matching ST symbols to their competitor counterparts is the part that, done manually, costs the most time and is the least repeatable, so it received the most care. Rather than rely on name similarity alone — which fails precisely when two vendors name the same function differently — the engine scores each candidate pair by combining twelve weighted signals into a single, explainable confidence value. Role similarity and alias similarity carry the most weight (0.18 each), followed by name similarity and operation similarity (0.12 each), external references and signature cues (0.10 each), footprint proximity (0.08), object similarity (0.06), context cues (0.04), and three knowledge-base priors (history, board-pair, scenario — contributing the remaining 0.02). The weights sum to exactly 1.0. Figure 2.6 shows the relative weights, and Figure 2.7 expands the computation and penalty mechanics of each signal.

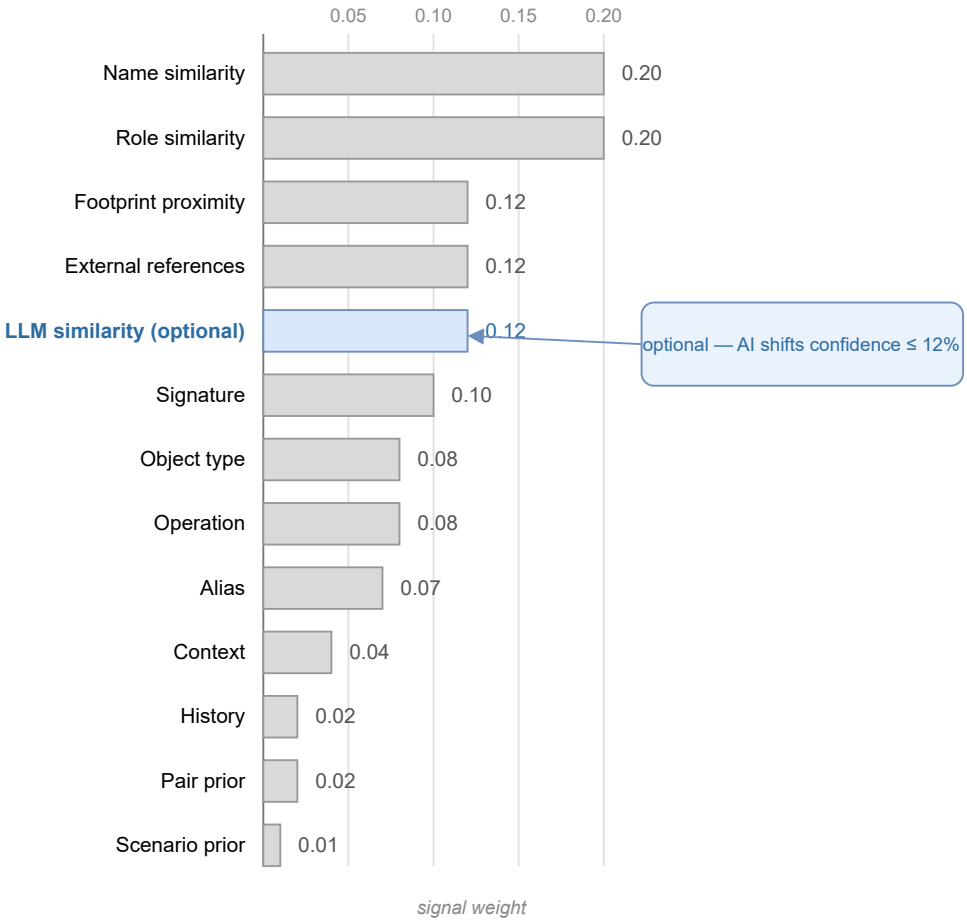

Figure 2.6: Relative weights of the signals combined by the deterministic mapping scorer. Name and role similarity dominate; the optional LLM-similarity signal (highlighted) is capped so that AI assistance can shift a candidate's confidence by at most twelve percent.

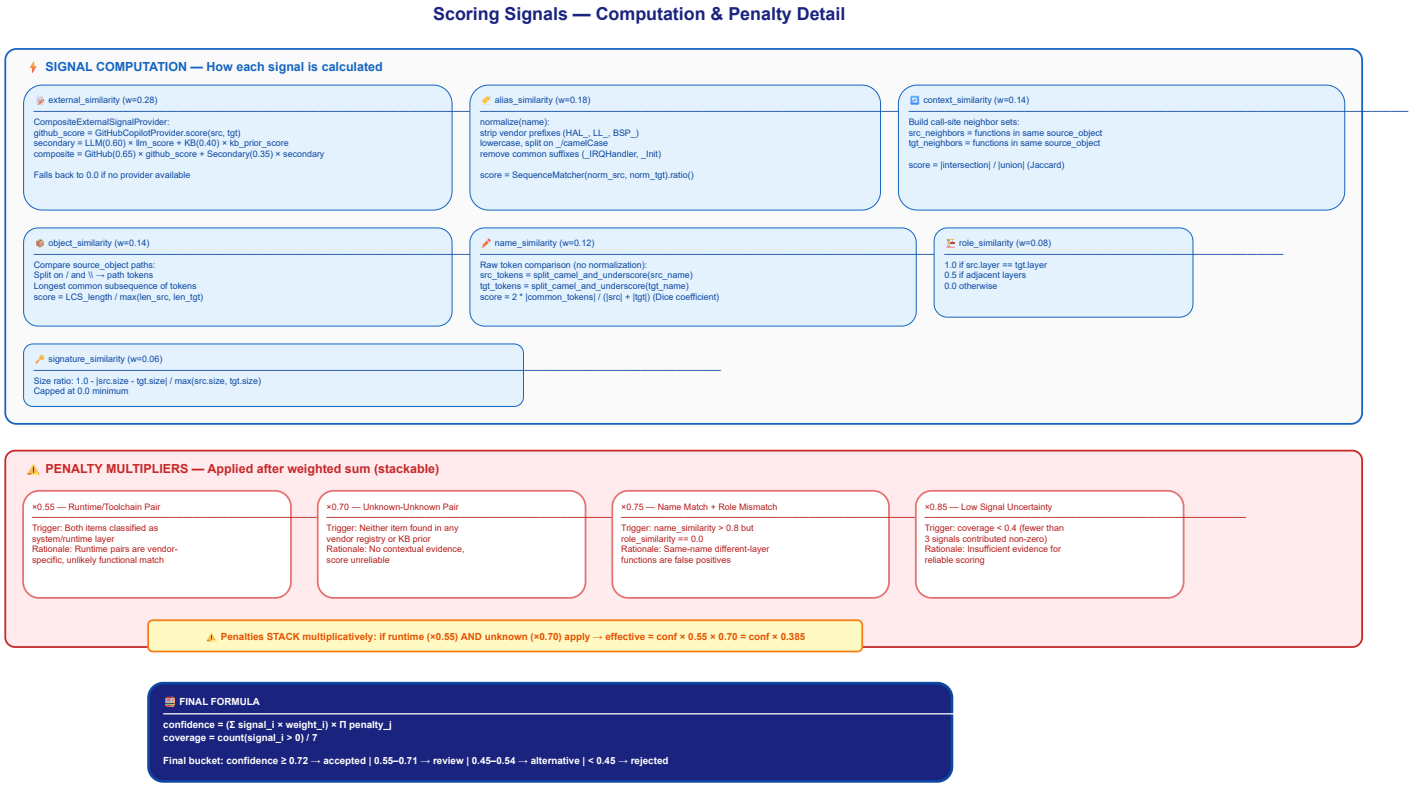

Figure 2.7: Detailed signal computation and penalty system. Seven evidence signals are computed for each candidate pair; penalties for size mismatch and layer conflict reduce the composite score before the acceptance threshold is applied.

The resulting confidence drives a transparent acceptance policy. A correspondence is accepted automatically once its confidence reaches 0.72, queued for manual review above 0.55, kept as a weaker alternative above 0.45, and an unmatched symbol is retained on its own only above 0.80. The engine also records the cardinality of each match — one-to-one, one-to-many, or many-to-one — which matters because vendors routinely split or merge functionality differently, and a one-to-many match is itself a finding about how the two stacks are structured. Every decision, with its per-signal breakdown, is written to the mapping diagnostics so that a reviewer can see exactly why two symbols were paired. Figure 2.8 illustrates the complete flow from candidate generation through pruning, scoring, and threshold-based bucketing.

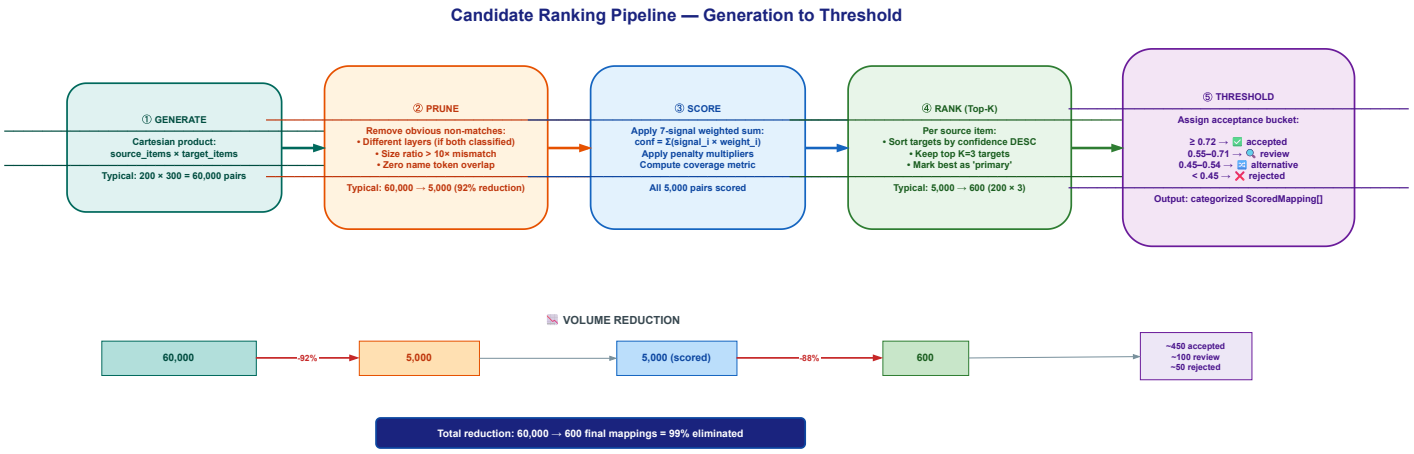

Figure 2.8: Candidate generation, ranking, and threshold flow. The Cartesian product of source and target items is pruned by fast rejection filters, scored on all signals, and bucketed into accepted, review, alternative, and unmatched categories by confidence thresholds.

2.4.3 Optional AI assistance

The tool is best described as a deterministic core with *optional* language-model assistance, not as an "AI tool" in the sense of one that cannot work without a model. Every language-model feature is opt-in and disabled by default, and the engine produces a complete, reproducible result with all of them switched off. When they are enabled, they act in three bounded ways. First, a large language model (LLM) can supply one additional similarity signal during mapping; as Figure 2.6 makes explicit, that signal is weighted and capped so it can move a candidate's confidence by at most twelve percent, and the run stays within a fixed call budget. Second, a "confident pass" may confirm or reject borderline matches, and an advisor may suggest where STM32 firmware could be tightened. Third, the desktop interface offers a chat assistant for exploring a comparison in natural language.

These features can be served by several interchangeable back-ends, listed in Table 2.3, selected automatically in a fixed order or pinned explicitly. They range from a hosted GitHub Copilot endpoint [16] to a fully offline Ollama server [19], so the assistance can run without any data leaving the machine. If a back-end is unavailable or returns something unusable, the tool degrades gracefully to its deterministic output rather than failing. Figure 2.9 shows the provider abstraction and the auto-detection fallback chain.

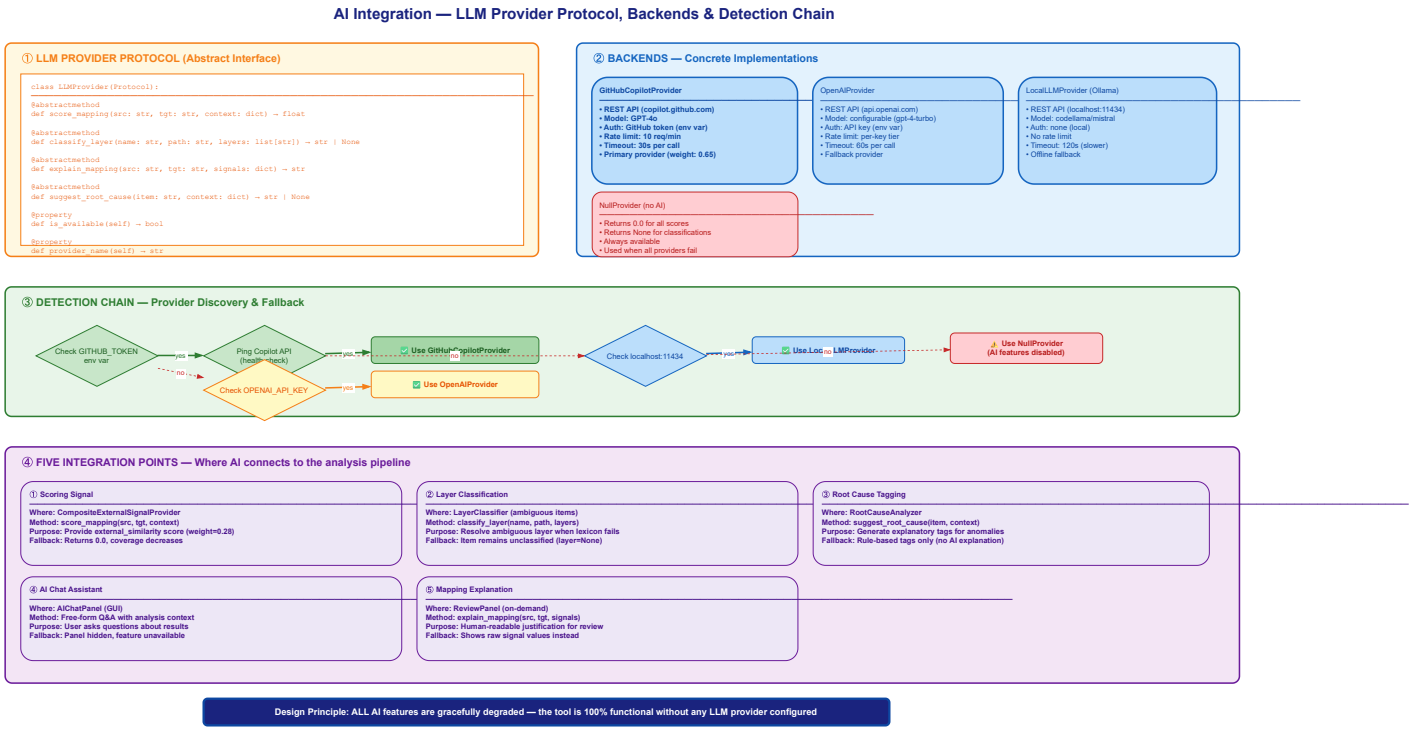

Figure 2.9: AI integration flow. A common protocol abstracts all back-ends; an auto-detection chain probes them in priority order (Copilot, OpenAI, Ollama) and falls back gracefully to purely deterministic operation if none is available.

Table 2.3: Optional language-model back-ends. All are disabled by default; the tool runs fully without them.

Back-end	**Default model**	**Selection / requirement**
GitHub Copilot [16]	Claude (Anthropic) [17]	OAuth device flow; first in the auto-detect order.
OpenAI [18]	GPT-4o	`OPENAI_API_KEY` environment variable.
Ollama [19]	Llama 3.1 (8B)	Local server; fully offline.
GitHub Models [20]	gpt-4.1-mini	Default for the narrative assistant.

2.4.4 Persistence and learning

All state lives in local SQLite databases: the vendor registry, the store of confirmed correspondences, a cache of assistant responses, and a history of runs. A manual confirmation made during review can be replayed on later runs, so the tool improves as it is used on a given vendor pair. We treat this learning, together with the web-ingestion and run-history features, as useful but experimental, and none of it is on the critical path of a comparison.

2.4.5 Cross-vendor equivalence database

A critical addition to the scoring engine is the *cross-vendor equivalence database*, which encodes deterministic knowledge about which modules and functions serve the same peripheral function across vendor SDKs. When two symbols are confirmed equivalents by this database, their confidence is boosted to the high-confidence threshold regardless of naming differences.

The database covers twenty-five peripheral groups (GPIO, UART, SPI, I^2C, Timer, Clock, Power, DMA, USB Device, USB Host, Flash, ADC, DAC, CAN, Ethernet, RTC, Watchdog, CRC, RNG, I^2S, Pin Mux, Interrupt, Cortex core, BSP/Board, System Init, Startup, and PWM) with token-pattern entries for six vendors: STM32, NXP, Renesas, TI, GigaDevice, and Infineon. It also encodes sixteen action-verb equivalence classes (init/deinit, start/stop, transmit/receive, set/get, reset, callback, register/ unregister, lock/unlock, suspend/resume), so that `HAL_GPIO_Init` and `GPIO_Setup` are recognised as serving the same role.

The database is designed as an offline, provenance-tracked artefact: an equivalence miner extracts entries from pinned vendor SDK repositories (resolved to a specific commit SHA for reproducibility), records provenance for every entry, and produces a content-hashed mining manifest. This ensures that the equivalence knowledge used in any given run is auditable and frozen — it cannot drift silently between executions.

2.4.6 Source-code signal provider

When both source and target IAR EWARM project directories are available, an optional source-code signal provider enriches the mapping with three additional signals derived from the actual C source files: function signature similarity (parameter count and return-type patterns), call-graph structural similarity (shared callee patterns), and include-graph co-location (functions in files with similar `#include` dependencies). This provider operates as an external-signal plug-in to the scoring engine and is disabled by default; when enabled, it provides a richer correspondence basis for firmware that is available locally, without requiring any network access or language model.

2.4.7 Firmware quality metrics and grading

Beyond comparing two builds, the tool computes firmware quality metrics for the source (STM32) firmware independently: total flash and RAM, module and function counts, average and maximum module size, layer-balance score, modularity score, code density, HAL abstraction ratio, middleware

ratio, application ratio, large-module count, dead-weight estimate, optimisation potential, and an overall quality grade (A through F). These metrics appear in the PDF report and help the technical marketing team assess not just how STM32 compares to a competitor, but how well the STM32 firmware itself is structured.

2.4.8 Run manifest and reproducibility

Every benchmark run emits a *run manifest* that captures the tool version, the deterministic mode (strict or assisted), the schema version, input file checksums, and a hash of the equivalence database state at the time of execution. This manifest allows any run to be proven reproducible: given the same inputs, configuration, and frozen knowledge state, the tool must produce byte-for-byte identical logical output. The deterministic hardening specification formalises this guarantee by requiring stable sort tie-breaks, rounded floating-point scores, and canonical export ordering across all stages.

2.5 Integration into the benchmark workflow

The tool occupies a precise place in our method. Once a software stack has been built for an STM32 board and for the competing vendor's board under identical scenarios, each build's IAR linker map is fed to the tool as the source and the target. Its per-layer and global footprint figures, its verdict, and its root-cause breakdown are exactly the numbers the per-vendor benchmarking chapters of this report quote when they weigh ST's flash and RAM cost against a competitor's for a given stack. In other words, the footprint comparisons reported later are not assembled by hand: they are produced, with an auditable trail, by the tool described here, which is why we document it before presenting any results that depend on it.

2.6 Usage and outputs

[PLACEHOLDER — GUI screenshot]

Capture the MCU Workflow Studio desktop interface showing a completed benchmark comparison (main window with charts and layer breakdown).

Figure 2.10: MCU Workflow Studio desktop interface showing a USB HID footprint comparison between STM32 and a competitor.

A comparison can be driven either from the desktop interface or, for reproducible and scriptable runs, from the command line. A typical invocation names the two maps and their vendors and points at an output directory, as in Listing 2.1.

```
mcu-workflow-cli benchmark \
  --source stm32usbhidclassproject.map --source-vendor stm32 \
  --target nxpusbhidclassproject.map   --target-vendor nxp \
  --out results/
```

Listing 2.1: Command-line invocation of a footprint benchmark.

The run leaves behind the artefacts of Table 2.1: a results workbook with the per-layer and global comparison, scoped CSV/JSON exports, a prioritised review queue of the matches that warrant a human look, the full mapping diagnostics, and — on request — an eight-page PDF report with an executive summary, source-versus-target charts, an architecture-and-module breakdown, a gap analysis, and recommendations. The samples we used to develop and validate the tool are themselves a representative comparison: a USB human interface device (HID) class project built for an STM32U575 board against

the equivalent project built for an NXP MCXN947 board [21], both compiled with the same IAR toolchain so that only the vendor software stacks differ.

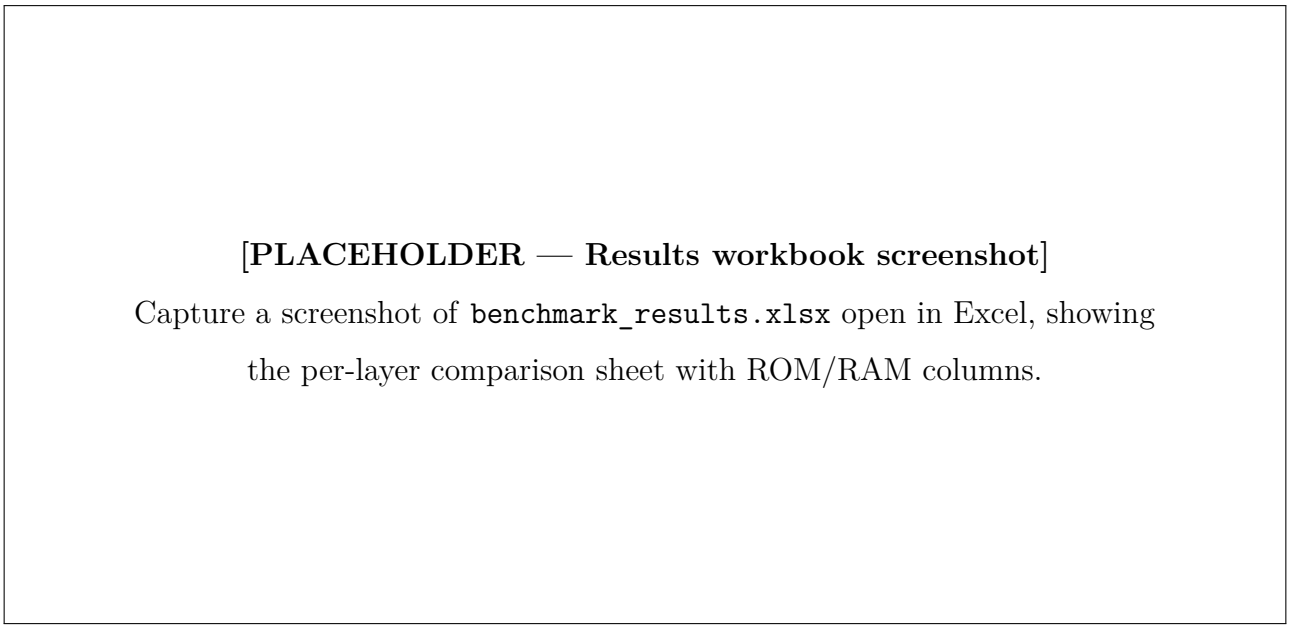

Figure 2.11: Sample output: the results workbook showing per-layer and global footprint comparison.

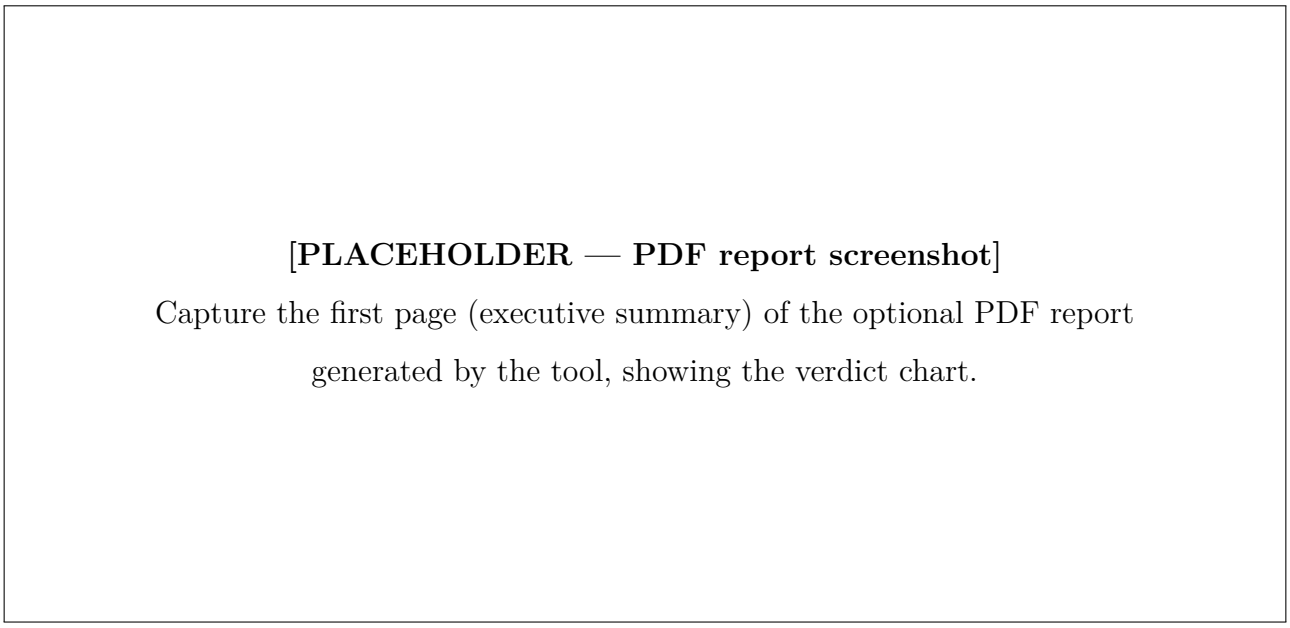

Figure 2.12: First page of the optional PDF report: executive summary with source-vs-target charts.

2.7 Engineering value and limitations

The value of the tool is that it turns a slow, subjective, once-off comparison into a fast, explainable, and repeatable one. A footprint benchmark that previously took an afternoon of careful map-reading

now runs in seconds and produces the same answer every time, with a documented reason behind every symbol correspondence and a clear attribution of where each kilobyte of difference originates. The run manifest, the frozen equivalence database, and the deterministic hardening guarantee that the same inputs always yield the same verdict — satisfying the reproducibility requirement of academic evaluation.

The twelve-signal scoring engine, backed by the cross-vendor equivalence database covering twenty-five peripheral groups and sixteen action-verb classes, achieves high-confidence mappings across six vendors without relying on language models. The firmware quality metrics and grading system provide an independent structural assessment of the STM32 firmware, and the multi-run history tracks how footprint evolves across SDK versions.

We are equally clear about what the tool does not do. It reads IAR EWARM linker maps; builds produced by other toolchains are out of scope for now, and broadening the parser is the most obvious avenue for future work. Its footprint figures are static, derived from the linked image rather than from execution. The quality grades and optimisation-potential estimates that the report produces are heuristic indications, not measured quantities, and we label them accordingly wherever they appear. Finally, the language-model features, while useful, are optional and still maturing; the trustworthy core of the tool is, and is meant to remain, fully deterministic.

Conclusion

We designed and built MCU Workflow Studio to remove the manual bottleneck in cross-vendor footprint benchmarking and to make its results reproducible and explainable. Its deterministic core parses two IAR linker maps, classifies every symbol into an architectural layer, maps STM32 symbols to their competitor counterparts using a twelve-signal scoring engine backed by a cross-vendor equivalence database, and reports per-layer and global flash and RAM differences with a clear root cause. The run manifest guarantees reproducibility; the firmware quality metrics provide an independent structural assessment; and the optional, strictly bounded language-model assistance enriches the mapping without ever sitting on the critical path. The chapters that follow rely on this tool for the footprint figures they report, which is why we have presented it first, as a contribution in its own right.

Chapter 3

An Autonomous AI Agent for Benchmark Campaigns

Introduction

The previous chapter presented a tool that analyses the *output* of a benchmark — two linker maps produced by completed builds. However, producing those builds in the first place is itself a substantial engineering task. Each new cross-vendor comparison requires creating a benchmark project for the competitor platform, porting the STM32 reference code while preserving its exact runtime semantics, configuring the IAR Embedded Workbench for ARM (EWARM) project correctly, and debugging the inevitable compiler, linker, and runtime issues that arise when targeting unfamiliar silicon. Done manually, this process is slow, error-prone, and difficult to keep consistent across many vendor comparisons.

To address this, we designed and built the *MCU Benchmark Copilot Agent* — a custom GitHub Copilot agent system that operates inside Visual Studio Code (VS Code) and *autonomously* drives end-to-end benchmark campaigns: from scenario resolution and firmware acquisition, through project creation, IAR build automation, evidence-driven debugging, measurement extraction, and semantic-equivalence verification [16, 22]. Unlike a passive assistant, the agent executes builds, fetches missing materials from official vendor sources, applies fixes, and iterates — asking the user only before programming physical hardware. Where MCU Workflow Studio analyses what *exists*, the agent system *creates* what does not yet exist, and it does so under strict rules that enforce semantic equivalence and benchmark fairness at every step. The two tools are complementary: the agent produces validated builds, and the studio analyses their footprint.

This chapter documents the agent as an engineering contribution. We describe the problem it

addresses, its layered architecture (agents, skills, automation tools, configuration registry), its deterministic campaign workflow, the always-on policies that govern its behaviour, and its integration with the broader benchmarking effort.

3.1 The benchmark engineering problem

A single STM32-versus-competitor benchmark involves several non-trivial steps. The STM32 reference project must first be understood in full — task structure, timing, peripheral usage, reporting format, and observable behaviour. A corresponding project must then be created for the target platform, mapping every STM32 hardware abstraction layer (HAL) call to the competitor's equivalent application programming interface (API) without altering the benchmark's semantics. The resulting project must be configured for the IAR toolchain [15] with the correct device selection, startup file, linker configuration, include paths, and compiler defines. Once it compiles, runtime issues — wrong interrupt vectors, misconfigured clocks, broken universal asynchronous receiver-transmitter (UART) output — must be diagnosed from real evidence rather than guesswork. And throughout, the comparison must remain fair: same optimisation level, same measurement scope, same observable output format.

Doing this for one vendor pair and one software stack is manageable. Repeating it for multiple vendors (GigaDevice, NXP, Renesas, Texas Instruments, Infineon, Nordic, Microchip) across multiple stacks (USB device, FreeRTOS, CoreMark) quickly becomes the dominant time cost of the benchmarking campaign. Each port also carries a risk of *semantic drift* — subtle changes in task priorities, timing, or synchronisation that invalidate the comparison without being immediately visible.

We needed a system that could enforce the methodology consistently across every port, refuse to guess when evidence was missing, execute deterministic build and measurement workflows without manual intervention, and make its reasoning transparent. A general-purpose large language model (LLM) cannot do this out of the box: it lacks the domain-specific checklists, the structured routing, the automation tooling, and the policy of refusing to claim success without hardware evidence. A custom agent system, built on VS Code's Copilot extensibility framework, gave us the ability to encode our benchmarking methodology — and to *execute* it autonomously — directly within the development environment [23].

3.2 Architecture

The system is structured as four cooperating layers: specialist *agents* that handle reasoning and delegation, reusable *skills* that encode methodology as checklists, *automation tools* (PowerShell and Python scripts)

that perform deterministic operations, and a *configuration registry* (YAML files) that provides ground truth about boards, MCUs, vendors, toolchains, and measurement profiles. A shared set of always-on rules governs all layers. All configuration lives in a `.github/` directory within the benchmark workspace — no compiled code, no external server, no deployment step. The agents run inside VS Code's GitHub Copilot Chat extension [16] and have access to the workspace file system, terminal, web, and search capabilities.

Figure 3.1 illustrates the layered arrangement. A user request enters the coordinator agent, which resolves the scenario, classifies the task, acquires missing materials, and delegates to the appropriate specialist agent or invokes a skill workflow. Specialist agents operate in isolated context, use the automation tools for deterministic operations, and return structured results.

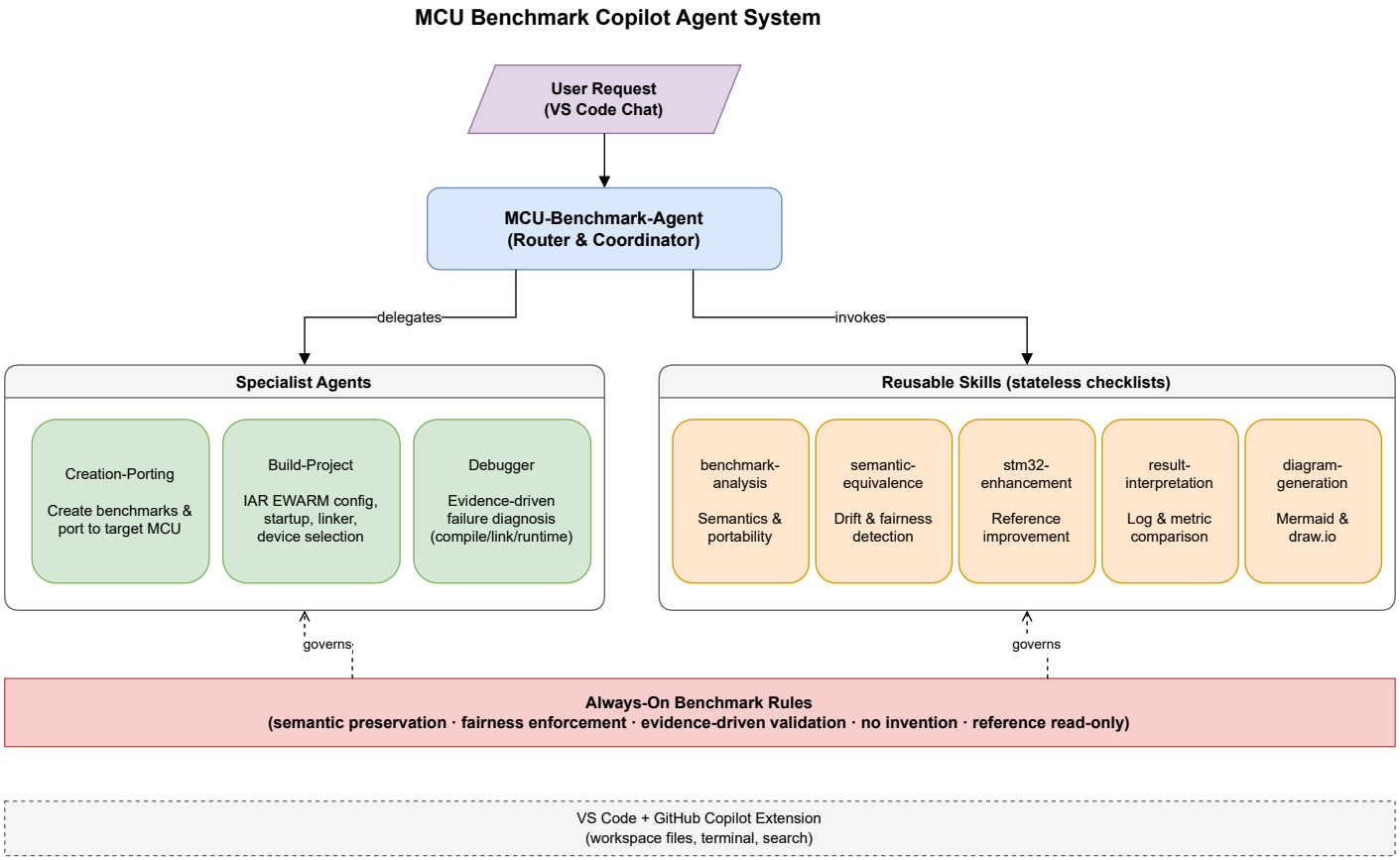

Figure 3.1: Architecture of the MCU Benchmark Copilot Agent system. The coordinator resolves scenarios and delegates to specialist agents; automation tools handle deterministic build/measurement operations; all layers operate under the shared always-on benchmark rules and consult the configuration registry.

3.2.1 The four specialist agents

Each agent has a defined responsibility, a restricted tool set, an autonomy contract, and a structured output contract. Table 3.1 summarises them.

Table 3.1: Specialist agents and their responsibilities.

Agent	Role	Responsibility
MCU-Benchmark-Agent	Autonomous coordinator	Resolves scenarios from partial input, acquires missing materials, orchestrates end-to-end campaigns, delegates to specialists. Full tool access including web and execute.
Creation-Porting	Project generation	Creates new benchmark projects from scratch or ports existing benchmarks in any direction (STM32→competitor, competitor→STM32, vendor→vendor, board→board). Acquires firmware autonomously.
Build-Project	Build automation	Autonomously executes IAR builds, classifies failures, applies project-level fixes, and rebuilds — without user intervention for non-hardware tasks.
Debugger	Failure diagnosis	Autonomously diagnoses compiler, linker, and runtime failures; extracts measurements from logs and captures; applies minimal fixes and revalidates.

The coordinator's routing decision tree is deterministic: project understanding goes to the *benchmark-analysis* skill; cross-vendor porting goes to *Creation-Porting*; build execution and configuration go to *Build-Project*; failures accompanied by real evidence go to *Debugger*; and result interpretation uses the dedicated skill. This routing prevents a general-purpose model from attempting tasks outside its configured expertise and ensures each specialist receives only the context it needs.

Figure 3.2 presents a complete use case diagram of the system, showing all actors, the twenty internal capabilities, and the hardware approval boundary that separates autonomous operations from those requiring user confirmation.

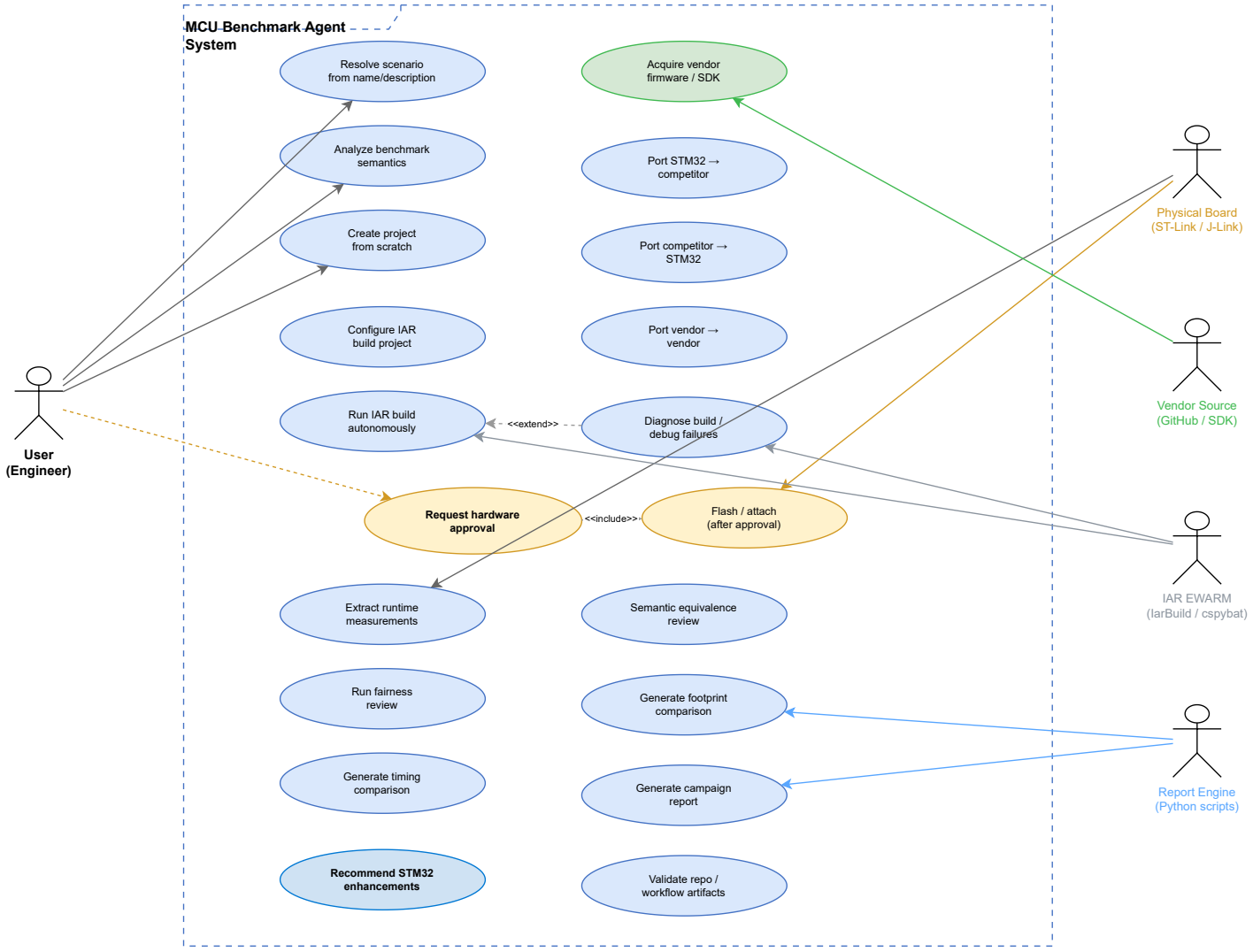

Figure 3.2: UML use case diagram of the MCU Benchmark Copilot Agent system. Blue ellipses represent autonomous use cases; yellow ellipses require user approval; green indicates external acquisition. External actors include the user, physical board, vendor source repositories, IAR EWARM toolchain, and the report engine scripts.

3.2.2 The six reusable skills

Skills encode repeatable workflows as structured Markdown checklists. Unlike agents, they do not receive isolated context or delegated reasoning — they provide structure, not autonomy. Table 3.2 lists them.

Table 3.2: Reusable skills and their purposes.

Skill	Purpose	Primary users
benchmark-analysis	File classification, semantics extraction, and portability split of an existing project.	Coordinator, Creation-Porting
semantic-equivalence	Reference-versus-target behaviour comparison and drift detection.	Coordinator, Creation-Porting
iar-template-resolution	Resolve, acquire, and integrate IAR project templates per board type using a deterministic lookup algorithm.	Creation-Porting, Build-Project
stm32-enhancement	Gap-driven recommendations for improving STM32 reference portability.	Coordinator
result-interpretation	Comparative interpretation of benchmark logs, measurements, and result files.	Coordinator, Debugger
diagram-generation	Mermaid and draw.io workflow, architecture, and presentation diagrams.	Coordinator

3.2.3 Automation tools

A critical addition to the system is a library of twelve deterministic scripts (`.github/tools/`) that the agents invoke via the terminal for operations that must produce repeatable results. Table 3.3 lists the key scripts.

Table 3.3: Automation tools in the agent system.

Script	Language	Purpose
`run_iar_build.ps1`	PowerShell	Wraps IarBuild.exe with structured output, exit-code handling, and log capture.
`discover_iar.ps1`	PowerShell	Deterministic IAR installation discovery at known paths.
`acquire_iar_template.ps1`	PowerShell	Downloads and materialises IAR board templates from official vendor sources.
`run_cspy_capture.ps1`	PowerShell	Runs cspybat.exe batch debug sessions and captures measurement output [25].
`collect_measurements.py`	Python	Parses UART, SWO, and C-SPY output into structured measurement YAML.
`normalize_measurements.py`	Python	Normalises raw measurements for cross-platform comparison.
`compare_measurements.py`	Python	Computes deltas and fairness-aware comparisons between platforms.
`normalize_build_log.py`	Python	Structures IAR build diagnostics into classified error/warning lists.
`parse_map_file.py`	Python	Extracts memory-usage data from IAR linker map files.
`generate_benchmark_report.py`	Python	Produces structured campaign reports from collected artifacts.
`validate_benchmark_repo.py`	Python	Validates workspace structure and campaign artifact completeness.
`validate_iar_templates.py`	Python	Checks materialized IAR templates against expected file inventories.

These scripts ensure that build commands, measurement parsing, and comparison logic execute identically regardless of which agent invokes them or when. The agents handle reasoning and decision-making; the scripts handle deterministic execution.

3.2.4 Configuration registry

The YAML-based configuration registry (`.github/config/`) provides ground truth that the agents consult rather than hallucinate. Table 3.4 lists its files.

Table 3.4: Configuration registry files.

File	Content
`boards.yaml`	Board registry with official identifiers, MCU, core, probe, measurement capabilities, and aliases for all supported evaluation boards.
`mcus.yaml`	MCU specifications: flash, RAM, clock, peripherals, startup and linker file references.
`vendors.yaml`	Vendor metadata: supported families, HAL framework, RTOS defaults, wrapper model, and board naming conventions for ST, TI, NXP, Renesas, GigaDevice, Infineon, Nordic, and Microchip.
`toolchains.yaml`	IAR EWARM binary paths, discovery order, build-command patterns, and configuration areas.
`iar_templates.yaml`	IAR project template registry with acquisition status, source URLs, and resolution algorithm metadata.
`vendor_sources.yaml`	Official firmware acquisition URLs and SDK references per vendor.
`measurement_profiles.yaml`	Standard measurement definitions (task-switch cycles, IRQ latency, flash/RAM footprint) with fairness risks and required evidence.
`reporting.yaml`	Output format templates for campaign reports.

The registry solves a fundamental problem with LLM-based agents: when the model does not know a board's pin mapping or a vendor's SDK repository URL, it might invent one. By encoding verified facts in YAML and instructing the agents to consult these files, we eliminate an entire class of hallucination errors.

3.3 Supported workflow directions

A key design decision was to support benchmarking in *all* directions, not only the obvious STM32-to-competitor port. The system handles six workflow directions:

1. **STM32 → competitor** — port an STM32 reference benchmark to a competitor target (the primary use case).
2. **Competitor → STM32** — port a competitor's reference project to an STM32 target (useful when the competitor has a ready example that ST lacks).
3. **Vendor A → Vendor B** — port between any two non-ST vendors.
4. **Board A → Board B** — replicate a ready scenario on a different board (same or different vendor).
5. **Scenario description → new project from scratch** — create a complete benchmark project from only a textual scenario description, with no pre-existing reference code.
6. **Firmware/benchmark name only → acquire and build** — the user provides only a firmware package name (e.g. `USB_Device_HID`); the agent resolves the scenario, acquires the firmware from official sources, and proceeds autonomously.

In all directions, the same semantic preservation and fairness rules apply. The agent generates a new target project rather than modifying any reference, and it preserves benchmark semantics (observable behaviours), not vendor API names.

3.4 Autonomy model and approval gates

The agent operates autonomously by default for all workspace inspection, file generation, build execution, debug preparation, material acquisition, and measurement extraction. Table 3.5 clarifies the boundary.

Table 3.5: Autonomy model: autonomous actions versus approval-required actions.

Autonomous (no confirmation)	**Requires user approval**
Read, search, and inspect any workspace file	Flash or program a physical board
Create, edit, or reorganise application-layer project files	Erase or reset a physical device
Run IarBuild.exe and other local build commands	Attach a live debug probe to real hardware
Parse build logs, linker maps, UART output, and debugger captures	Modify vendor baseline file contents (startup, linker, HAL, BSP, middleware)
Fetch firmware, CMSIS headers, BSP, and documentation from official vendor sources	—
Apply minimal project-configuration fixes and rebuild	—
Run cspybat.exe in simulation/batch mode	—

After the user grants one explicit board-programming approval in a session, the agent continues autonomously through the remaining flash → debug → measure → report sequence for that board without re-asking. This design maximises throughput while maintaining a hard safety boundary at the point where actions become physically irreversible.

3.5 Board firmware baseline immutability

A distinctive policy of the system is the *Board Firmware Baseline Immutability Rule*. Vendor SDK, HAL, Low-Layer (LL), Board Support Package (BSP), startup, linker, middleware, and driver files constitute a read-only baseline infrastructure. The agents integrate benchmarks exclusively at the application layer, preserving the board firmware baseline intact. Table 3.6 classifies the writable and protected surfaces.

Table 3.6: Surface classification under the baseline immutability rule.

Surface	**Policy**	**Examples**
Application layer	Writable (default)	`main.c`, `benchmark_tasks.c`, app wrappers and adapters
Middleware / RTOS	Protected (read-only)	FreeRTOS kernel, USB stack, filesystem
HAL / BSP / Startup / Linker	Protected (read-only)	`startup_*.s`, `*.icf`, `stm32xx_hal_*.c`
Build surface	Configure only	`.ewp` references, include paths, defines, device selection

The key distinction is between *selection* and *modification*. The agent may freely *select* which existing vendor file to reference in the project (e.g. choosing `startup_stm32u575xx.s` from the SDK). However, *editing the content* of that file — to add instrumentation, change the vector table, or alter initialisation sequences — is an approval-required deviation. If a benchmark cannot be implemented through application-layer integration alone, the agent reports the need and requests explicit approval before proceeding. Any approved deviation is recorded in the campaign's `fairness_baseline.yaml` for audit.

This rule ensures that benchmark results reflect the vendor's actual firmware infrastructure, not a customised version that would invalidate the comparison.

3.6 Always-on benchmark rules

All agents and skills inherit a shared policy layer — the *always-on benchmark rules* — that constrains the system's behaviour regardless of which specialist is active. These rules encode the core principles of our benchmarking methodology:

- **Reference is read-only.** The STM32 reference project is never modified unless the user explicitly requests it; new target projects are generated instead.
- **Semantic preservation.** A seventeen-point checklist governs what must be identical between reference and target: task count and creation order, priorities, delays, periodicity, synchronisation primitives, suspend/resume and yield behaviour, counters, state transitions, measurement scopes, reporting format, and error behaviour.

- **No invention.** The system does not invent vendor APIs, pin mappings, peripheral choices, or board details. Uncertain hardware details are marked as `TODO` rather than guessed. The configuration registry is consulted for verified facts.
- **Evidence-driven validation.** No claim of runtime success is made without actual compiler output, runtime logs, UART captures, or debugger observations.
- **Fairness enforcement.** A fairness checklist — covering optimisation level, clock frequency, memory configuration, RTOS wrapper differences, measurement instrumentation overhead, and environmental differences — is applied to every comparison. Any fairness-sensitive difference is reported *before* performance conclusions.
- **Provenance tracking.** Every acquired resource (firmware, CMSIS headers, BSP files, documentation) is recorded with its source URL, version, branch/tag/commit, and acquisition date.
- **Command transparency.** Every command the agent executes (build, debug, inspection) is reported with its exact invocation, working directory, and outcome.

These rules solve a specific problem with using general-purpose LLMs for embedded work: without them, a model may silently simplify a benchmark, invent an API that does not exist, claim that code works when it has never been compiled, or obscure the provenance of acquired materials. The always-on policy prevents all four failure modes.

3.7 Semantic preservation in detail

The semantic-equivalence skill deserves elaboration because it is the mechanism that keeps cross-vendor comparisons valid. When the Creation-Porting agent produces a target project, the semantic-equivalence skill is invoked to compare the target against the reference on every point of the checklist. The result is classified into one of five categories:

1. **Equivalent** — behaviour matches within the benchmark definition.
2. **Equivalent with caveats** — behaviour matches, but environment or implementation details affect fairness (e.g. a different tick source).
3. **Drift** — behaviour differs and may affect results.
4. **Approval required** — an intentional deviation that the user must accept.

5. **Unknown** — evidence is insufficient to determine equivalence.

Any result other than category 1 halts the workflow and produces a clear report of what differs and why. This prevents semantic drift from silently propagating into published benchmark numbers.

3.8 Deterministic campaign workflow

Every autonomous benchmark campaign follows a stable twelve-step sequence. This determinism ensures that results are reproducible regardless of when or how many times the campaign is executed.

1. **Resolve scenario** — identify benchmark, board, MCU, and firmware from partial user input and workspace context.

2. **Acquire materials** — fetch missing firmware, startup, linker, CMSIS, BSP, and device descriptions from official sources; record provenance.

3. **Analyse semantics** — extract exact observable behaviours, identify portable versus board-specific layers (invoke *benchmark-analysis* skill).

4. **Resolve IAR template** — look up the target board in the template registry; acquire and materialise if needed (invoke *iar-template-resolution* skill).

5. **Create or port project** — delegate to Creation-Porting agent; generate new target project at the application layer only.

6. **Configure build** — delegate to Build-Project agent; set device, debugger, startup, linker, defines, includes consistently.

7. **Build** — execute IarBuild.exe autonomously; classify failures; fix and rebuild.

8. **Debug preparation** — delegate to Debugger agent; parse logs, apply minimal fixes, revalidate.

9. **Hardware approval gate** — ask user before flashing/attaching to real board.

10. **Extract measurements** — capture runtime metrics from UART, debugger, or batch sessions.

11. **Semantic equivalence and fairness review** — verify that the port preserves all benchmark-observable behaviours; report all fairness-sensitive differences.

12. **Generate report** — produce structured output with provenance, results, fairness status, and unresolved unknowns.

Figure 3.3 shows the end-to-end lifecycle with the handoff point to MCU Workflow Studio.

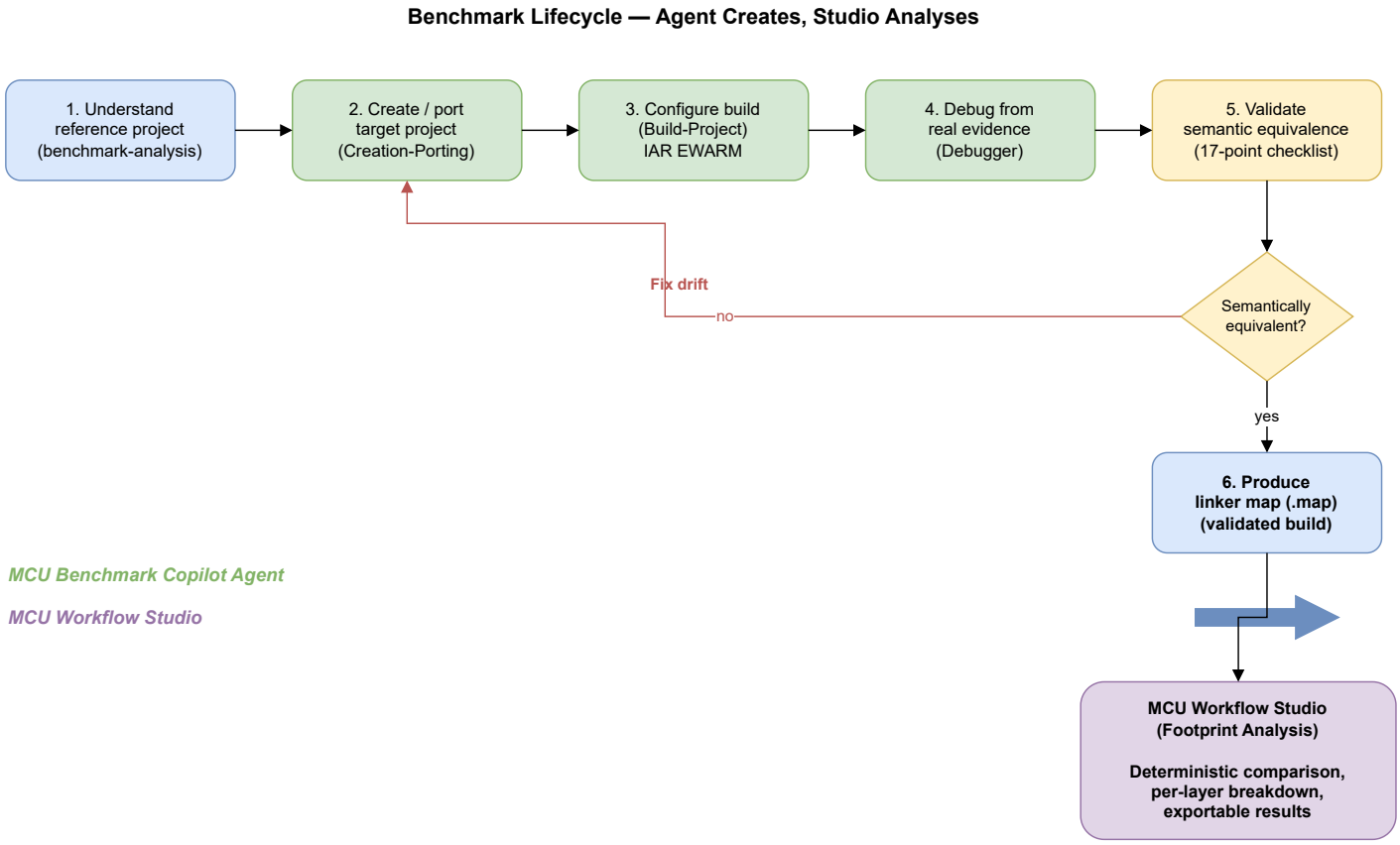

Figure 3.3: End-to-end benchmark lifecycle. The MCU Benchmark Copilot Agent handles steps 1 through 11 autonomously (with a hardware gate at step 9); once semantic equivalence is confirmed, the validated linker map is handed to MCU Workflow Studio for footprint analysis.

Figure 3.4 presents the same campaign as a UML sequence diagram, showing the actual participant delegation, message flow, the hardware approval gate, and artifact emission at each step.

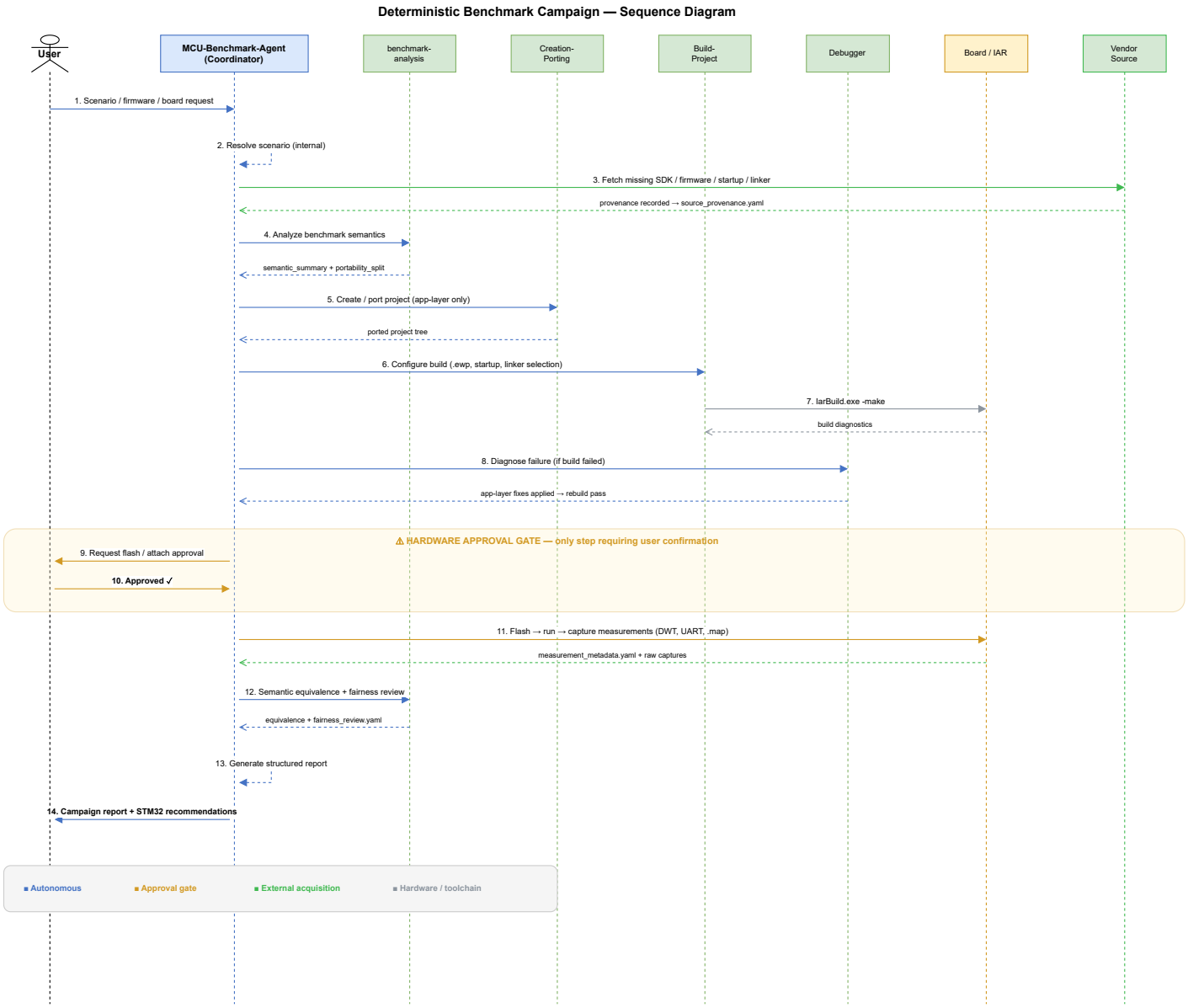

Figure 3.4: UML sequence diagram of a deterministic benchmark campaign. The coordinator delegates to specialist agents (benchmark-analysis, Creation-Porting, Build-Project, Debugger) and interacts with external actors (Vendor Source, Board/IAR). The hardware approval gate (steps 9–10, highlighted) is the only point requiring user confirmation; all other steps execute autonomously.

This separation keeps each tool focused. The agent does not attempt footprint analysis, and the studio does not attempt code generation or debugging. Together, they cover the full lifecycle of a cross-vendor benchmark: resolve, acquire, create, build, debug, validate, measure, then analyse.

3.9 IAR template resolution

Before creating any benchmark project, the system must resolve the correct IAR project template for the target board. We designed a deterministic four-step resolution algorithm encoded in the *iar-template-*

resolution skill:

1. **Exact board match** — search the template registry (`iar_templates.yaml`) for an entry whose `board_id` matches the target exactly.
2. **Family fallback** — if no exact match, search for a family-level fallback template and adjust device, startup, and linker for the specific MCU.
3. **Board-type fallback** — if no family match, select the closest template matching the board type and vendor with explicitly downgraded confidence.
4. **Unresolved** — if no suitable template exists, document the gap and report to the user rather than fabricating a template.

When a template is identified but not yet materialised locally, the `acquire_iar_template.ps1` script fetches it from official vendor GitHub repositories or SDK packages, records provenance, and verifies the file inventory. The materialised template then serves as the read-only project baseline into which benchmark application code is integrated through designated integration points.

3.10 Source acquisition policy

When materials are missing, the agent acquires them autonomously using a strict preference order:

1. Workspace files already present.
2. Local SDK content (IAR device/config directories, installed CMSIS packs).
3. Official vendor GitHub repositories (tagged releases preferred).
4. Official vendor websites, SDK packages, CMSIS Pack Index.
5. Third-party mirrors — only as a last resort, with an explicit risk note.

For every acquired resource, the agent records: source URL, version or branch/tag/commit, package name, and date of acquisition. This provenance chain ensures that any benchmark result can be traced back to the exact firmware version used, satisfying the reproducibility requirements of academic evaluation.

3.11 Campaign artifact model

Each benchmark campaign persists its state in a structured dossier under `.github/benchmark/campaigns/<camp`
Key artifacts include:

- `campaign_manifest.yaml` — campaign identity, platforms, and stage tracking.
- `execution_status.yaml` — per-stage status with timestamps and blockers.
- `source_provenance.yaml` — acquisition provenance for every resource used.
- `scenario_resolution.yaml` — how partial input was resolved.
- `fairness_baseline.yaml` — per-platform configuration baseline for fairness comparison.
- `semantic_equivalence.yaml` — verification results per checklist item.
- `fairness_review.yaml` — all fairness-sensitive differences identified.
- `measurement_metadata.yaml` — metrics collected, methods, triggers, and confidence levels.

This artifact model makes every campaign auditable and reproducible. A reviewer can inspect the dossier to understand exactly what was resolved, acquired, built, measured, and concluded — without re-running the campaign.

3.12 Prompt-driven usage

To lower the barrier to entry, the system ships ten pre-built prompt templates (`.github/prompts/`) that encode common benchmark tasks. The user selects a prompt from the Copilot chat picker and fills in minimal parameters (a board name, a scenario, or a firmware package name); the agent handles everything else. Table 3.7 lists the available prompts.

Table 3.7: Pre-built prompt templates for common benchmark tasks.

#	**Purpose**
01	Autonomous benchmark from firmware name only — agent resolves, acquires, builds, and measures.
02	Port a benchmark to another board with fairness guarantee.
03	Build, debug, and measure an existing project autonomously.
04	Compare two boards fairly with semantic-equivalence check.
05	Recommend STM32 improvements after a measured competitor gap.
06	Create a new benchmark from a scenario description (no reference).
07	Port STM32 reference to a competitor platform.
08	Port competitor benchmark to STM32.
09	Port between two non-ST vendors.
10	Resolve and build from a firmware/SDK package name alone.

[PLACEHOLDER — VS Code agent session screenshot]

Capture a VS Code window showing the Copilot Chat panel with the MCU Benchmark Agent active during an autonomous campaign (showing structured output with provenance, semantic equivalence, and fairness review).

Figure 3.5: An autonomous benchmark campaign in VS Code: the MCU Benchmark Copilot Agent resolves the scenario, acquires firmware, builds, and reports semantic equivalence and fairness status.

3.13 Implementation

The entire agent system is implemented within a `.github/` directory, leveraging the VS Code GitHub Copilot extensibility framework [16,23,24]. Agent and skill definitions use YAML frontmatter to declare metadata (name, description, tool permissions) and Markdown bodies for behavioural instructions. The automation tools are standalone PowerShell and Python scripts. Table 3.8 summarises the file structure.

Table 3.8: File structure of the agent system.

Path	Role
`.github/copilot-instructions.md`	Always-on rules inherited by every agent and skill.
`.github/agents/*.agent.md`	One file per specialist agent (4 files).
`.github/skills/*/SKILL.md`	One directory per skill, each containing a structured checklist (6 directories).
`.github/tools/*.ps1, *.py`	Twelve automation scripts for deterministic operations.
`.github/config/*.yaml`	Eight configuration registry files (boards, MCUs, vendors, toolchains, templates, sources, measurements, reporting).
`.github/prompts/*.prompt.md`	Ten pre-built prompt templates for common tasks.
`.github/templates/benchmark-dossier/`	Campaign artifact templates (YAML skeletons).
`.github/docs/`	Developer documentation (quickstart, porting guide, validation guide, naming conventions, etc.).
`.github/schemas/`	JSON/YAML schemas for artifact validation.

The implementation requires no external server, no API keys beyond the user's existing GitHub Copilot subscription, and no deployment pipeline. Adding a new agent, skill, tool, or board entry is a matter of creating a file and restarting the Copilot Chat session. This minimal footprint was deliberate: the system should be as easy to maintain and version-control as the benchmark source code itself.

3.14 Integration with the benchmarking workflow

The agent system occupies a precise position in the benchmarking pipeline. It sits *before* MCU Workflow Studio in the workflow: the agent resolves the scenario, acquires materials, creates and validates the target project, ensures it compiles under IAR EWARM, confirms that its runtime behaviour matches

the reference, and extracts measurements. Only then does the build's linker map become an input to MCU Workflow Studio for footprint analysis. The relationship is strictly sequential and complementary — the agent is responsible for *producing and validating* correct builds, and the studio is responsible for *measuring and comparing* them.

3.15 Engineering value and limitations

The agent system's primary value is *autonomous methodology enforcement at scale*. By encoding the semantic-equivalence checklist, the fairness checklist, the evidence-driven validation policy, the baseline immutability rule, and the provenance tracking requirement directly into the tool's behaviour, we ensure that every benchmark campaign follows the same discipline regardless of which vendor is targeted or how many campaigns run in parallel.

In practical terms, the agent reduces the time to produce a correct, validated target project from several hours of manual porting and debugging to an autonomous session requiring human intervention only at the hardware-programming gate. The deterministic workflow, the configuration registry, and the campaign artifact model make every result auditable and reproducible.

The structured output contracts — each agent returns objective, scenario resolution, actions executed, semantic equivalence status, fairness status, key findings, unresolved unknowns, and next action — make the work transparent and reviewable without re-executing the campaign.

We acknowledge several limitations. The system depends on the quality of the underlying language model; while the always-on rules and the configuration registry constrain its behaviour, they cannot prevent all model errors, particularly for vendor APIs that are poorly represented in the model's training data. The tool supports IAR EWARM 9.60.3 as the primary toolchain; extending it to other compilers (GCC, Keil) would require additional Build-Project configurations and toolchain registry entries. The evidence-driven validation policy means the system halts at the hardware gate — it cannot flash a board without explicit approval. This is by design: in embedded benchmarking, the hardware is the ground truth, and no amount of static reasoning can substitute for it.

Conclusion

We designed and built the MCU Benchmark Copilot Agent as an autonomous campaign coordinator that fills the engineering gap between defining a benchmark and having a validated, measured, and fairly compared result. Its four specialist agents cover the full lifecycle from scenario resolution to

measurement extraction; its six reusable skills encode our methodology as structured checklists; its twelve automation tools provide deterministic execution; and its configuration registry eliminates hallucination for known hardware facts. The always-on rules enforce semantic equivalence, benchmark fairness, baseline immutability, provenance tracking, and evidence-driven validation at every step.

Together with MCU Workflow Studio, the agent forms a complete autonomous toolchain for reproducible, explainable cross-vendor firmware benchmarking: the agent resolves, acquires, creates, builds, debugs, validates, and measures; the studio analyses the resulting footprint. The system supports eight MCU vendors, six workflow directions, and ten pre-built prompt templates — making it scalable to the full breadth of the STM32Cube competitive analysis campaign.

Chapter 4

Benchmarking ST against GD32

Introduction

GigaDevice Semiconductor is a Chinese fabless semiconductor company that produces the GD32 series of 32-bit microcontrollers [26]. The GD32 family is often positioned as a pin-compatible, drop-in alternative to STM32 parts, sharing the same Arm Cortex-M cores, similar peripheral sets, and comparable pricing. This close correspondence makes GD32 a particularly relevant competitor: engineers evaluating an MCU platform frequently shortlist both vendors, and the choice often comes down to measurable differences in software-stack quality rather than silicon specifications.

In this chapter we compare the STM32Cube ecosystem against GigaDevice's GD32 firmware libraries on two middleware stacks: the **USB device stack** (exercised through HID and CDC scenarios) and the **FreeRTOS** real-time operating system integration (exercised through basic, cooperative, and preemptive scheduling scenarios). For each stack we present the scenario design, per-scenario footprint and execution-time results, a per-layer ROM breakdown, and an interpretation of the trade-offs the data reveals.

All measurements follow the methodology described in Chapter 1: identical scenarios on both platforms, same compiler optimisation level, same clock frequency, and results extracted from IAR EWARM linker maps using the footprint analysis tool presented in Chapter 2.

4.1 Hardware platforms

Every scenario in this chapter runs on two boards hosting comparable Cortex-M33 parts: the STMicroelectronics **NUCLEO-U575ZI-Q** (STM32U575ZIT6Q) and GigaDevice's **GD32E507** evaluation board (GD32E507ZET6)

Both devices integrate an Arm Cortex-M33 core clocked at 160 MHz for these tests, ensuring that performance differences reflect software-stack efficiency rather than raw silicon speed. Table 4.1 contrasts their key characteristics. Figure 4.1 shows the two boards used in our test setup.

[PLACEHOLDER — Board photo]

Take a side-by-side photo of the NUCLEO-U575ZI-Q and the GD32E507 evaluation board.

Figure 4.1: The two evaluation boards used for ST-vs-GD32 benchmarking: NUCLEO-U575ZI-Q (left) and GD32E507 (right).

Table 4.1: Board characteristics: ST reference vs GigaDevice GD32.

Characteristic	**ST (STM32U575ZIT6Q)**	**GD32 (GD32E507ZET6)**
Core	Arm Cortex-M33	Arm Cortex-M33
Clock (test)	160 MHz	160 MHz
Flash	2 MB	512 KB
SRAM	786 KB	128 KB
USB	Full-Speed device	Full-Speed device
Security	TrustZone, AES, PKA, HASH	—
Toolchain / IDE	STM32CubeIDE / IAR EWARM	IAR EWARM

The STM32U575 belongs to the ultra-low-power STM32U5 series and offers substantially more flash and RAM, hardware security accelerators, and a richer peripheral set [28]. The GD32E507 targets a similar performance point at a competitive price but with fewer on-chip resources [27]. For our benchmarks, the relevant constant is that *both devices run the same core at the same frequency*, so footprint and execution-time differences are attributable to the vendor's software stack and HAL implementation.

4.2 USB device stack

The Universal Serial Bus (USB) device stack is a critical middleware component for any MCU that needs to communicate with a host PC. We benchmark ST's USB Device Library (part of the STM32Cube middleware) against GigaDevice's USB library, both operating in **Full-Speed** mode and running bare-metal (no RTOS).

4.2.1 Scenario design

We defined two scenarios that cover the most commonly used USB device classes:

1. **HID (Human Interface Device)** — a low-rate interrupt-transfer scenario in which the MCU acts as a USB mouse, sending periodic HID reports to the host. This exercises the enumeration path, descriptor handling, and interrupt-endpoint transfer logic.

2. **CDC (Communication Device Class)** — a higher-throughput bulk-transfer scenario in which the MCU exposes a virtual COM port. The host sends data to the device, which echoes it back. This exercises bulk IN/OUT transfers, line-coding negotiation, and buffering.

Each scenario was implemented identically on both platforms: same USB descriptors, same endpoint configuration (Full-Speed), same clock source (internal oscillator — HSI48 on STM32, IRC48M on GD32), and same core frequency of 160 MHz.

[PLACEHOLDER — IAR EWARM project screenshot]
Capture the IAR Embedded Workbench showing both the STM32 and GD32 USB HID projects side by side (project navigator and build configuration).

Figure 4.2: IAR EWARM project structure for the USB HID benchmark: STM32 reference (left) and GD32 target (right).

Table 4.2 summarises the scenarios and the metrics collected.

Table 4.2: USB device stack benchmark scenarios.

Scenario	What it exercises	Metrics
HID	Interrupt transfers, enumeration, descriptor handling	ROM/RAM (bytes), init/enum/TX time (ms)
CDC	Bulk transfers, virtual COM port, echo loopback	ROM/RAM (bytes), init/enum/TX/RX time (ms)

4.2.2 Results

4.2.2.1 HID scenario

Table 4.3 presents the total ROM and RAM footprint for the HID scenario.

Table 4.3: USB HID device footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM (total)	16 269	9 175	+77 %
RAM (total)	3 446	9 608	−65 %

The STM32 firmware consumes 77 % more ROM than GD32, but uses 65 % *less* RAM. The RAM advantage is substantial: the GD32 implementation requires 0x2000 bytes (8 KB) of stack and 0x2000 bytes of heap, whereas the STM32 configuration needs only 0x400 bytes (1 KB) of stack and 0x200 bytes (512 bytes) of heap.

Table 4.4 breaks down the ROM consumption by architectural layer.

Table 4.4: USB HID ROM breakdown by layer (bytes).

Layer	STM32U575	GD32E507	Difference
Application	2 580	1 942	+33 %
HAL drivers	10 426	820	+1 171 %
MW stack	2 859	5 743	−50 %

The HAL layer dominates the difference. On STM32, the HAL RCC module alone accounts for 4 560 bytes (44 % of total HAL ROM) and the combined HAL/LL USB driver adds another 4 988 bytes

(48 %). By contrast, GD32's HAL layer is minimal at 820 bytes because GigaDevice folds much of the peripheral-initialisation logic into the middleware itself. This architectural choice gives GD32 a smaller HAL at the cost of a larger middleware layer: the GD32 USB library ROM (5 743 bytes) is twice the size of ST's (2 859 bytes).

The STM32 application layer is 33 % larger because the STM32Cube USB library externalises device configuration into user files (`usbd_desc.c`, `usbd_conf.c`), accounting for 985 bytes (38 % of application ROM). On GD32, these configuration concerns are handled inside the middleware. This design gives STM32 users more configuration flexibility at the cost of a larger application footprint.

Table 4.5 shows the execution-time measurements.

Table 4.5: USB HID execution time (ms).

Operation	**STM32U575**	**GD32E507**	**Difference**
Initialisation	9.990	26.084	−61.7 %
Enumeration	213	216	−1.4 %
Data transmission	0.0093	0.0094	−0.7 %

The STM32 USB initialisation is 61.7 % faster than GD32's — a significant difference that reflects more efficient clock-tree configuration and peripheral bring-up in the STM32Cube HAL. Enumeration and data-transmission times are nearly identical, as expected: these phases are dominated by USB protocol timing rather than software overhead.

4.2.2.2 CDC scenario

Table 4.6 presents the total footprint for the CDC scenario.

Table 4.6: USB CDC device footprint (bytes).

Metric	**STM32U575**	**GD32E507**	**Difference**
ROM (total)	17 049	9 085	+88 %
RAM (total)	4 089	9 683	−57.7 %

The pattern mirrors HID: STM32 uses substantially more ROM (+88 %) but far less RAM (−57.7 %). The CDC scenario adds transmit and receive buffer management, which increases both vendors' footprints relative to HID, but the ratio remains consistent.

Table 4.7 provides the per-layer ROM breakdown.

Table 4.7: USB CDC ROM breakdown by layer (bytes).

Layer	STM32U575	GD32E507	Difference
Application	2 683	1 882	+42.5 %
HAL drivers	10 446	820	+1 174 %
MW stack	2 423	5 709	−57.5 %

The root cause is the same as in HID. The STM32 HAL RCC module (4 560 bytes) and the HAL/LL USB driver (5 006 bytes) together account for 92 % of the HAL ROM. The STM32 application layer is 42.5 % larger due to the externalised USB device configuration files (`usb_device.c`, `usbd_cdc_if.c`, `usbd_desc.c`, `usbd_conf.c`), contributing 1 164 bytes (43 % of application ROM). In return, the STM32 USB middleware library itself is 57.5 % smaller than GD32's, because GigaDevice bundles configuration and driver logic into its middleware layer.

Table 4.8 shows the execution-time measurements.

Table 4.8: USB CDC execution time (ms).

Operation	STM32U575	GD32E507	Difference
Initialisation	9.994	26.076	−61.7 %
Enumeration	218.2	218.5	−0.1 %
Data transmission	0.00988	0.01003	−1.5 %
Data reception	0.00986	0.00993	−0.7 %

Once again, initialisation is where STM32 dominates: 61.7 % faster than GD32. The remaining operations (enumeration, transmission, reception) are within 1.5 % of each other, confirming that steady-state USB throughput is protocol-bound rather than software-bound.

4.2.3 Interpretation

The USB benchmark reveals a clear architectural trade-off between the two vendors.

4.2.3.0.1 ROM footprint. STM32 consumes 77–88 % more total ROM than GD32 across both scenarios. The dominant contributor is the HAL layer, which is roughly twelve times larger on STM32. Two modules drive this cost: the **HAL RCC** clock-configuration driver (≈4 560 bytes) and the **HAL/LL USB** peripheral driver (≈5 000 bytes). GigaDevice achieves a much smaller HAL by embedding

peripheral-initialisation logic inside the middleware library rather than exposing it as a separate, reusable driver layer.

4.2.3.0.2 RAM footprint. The situation is reversed: STM32 uses 57–65 % less RAM. The GD32 implementation defaults to much larger stack and heap allocations (8 KB each vs ≤1.5 KB for STM32). This suggests that the GD32 USB library relies more heavily on dynamic allocation or deep call chains, whereas the STM32Cube library uses statically allocated buffers and a shallower call stack.

4.2.3.0.3 Execution time. STM32's USB initialisation is consistently 61.7 % faster across both HID and CDC. After initialisation, the two platforms perform within 1–2 % of each other, as the USB protocol itself (enumeration handshake, bulk/interrupt transfers) dictates the timing.

4.2.3.0.4 Design philosophy. STM32Cube separates concerns: HAL handles clocks and peripheral access, the middleware handles protocol logic, and the application owns configuration through user-editable files (`usbd_conf.c`, `usbd_desc.c`). This modular design gives developers explicit control and reusability at the cost of a larger total ROM. GigaDevice takes a more monolithic approach — smaller HAL, but a thicker middleware that bundles configuration internally, reducing ROM but also reducing the user's configuration flexibility.

4.2.3.0.5 Enhancement proposals. The data suggests two concrete areas where STM32Cube could reduce ROM without sacrificing functionality:

1. **HAL RCC optimisation.** The HAL RCC module accounts for ≈4.5 KB in every USB project regardless of the clocks actually configured. A conditionally compiled or link-time-optimised RCC driver that includes only the clock paths used would significantly narrow the HAL gap.
2. **HAL/LL USB driver consolidation.** The combined HAL and LL USB driver (≈5 KB) provides a comprehensive abstraction but may include code paths (e.g. High-Speed, OTG, DMA) that Full-Speed-only projects never execute. Finer compile-time selection of USB features could reduce this cost.

4.3 FreeRTOS integration

FreeRTOS is the most widely adopted real-time operating system for Cortex-M microcontrollers [29]. Both STMicroelectronics and GigaDevice ship FreeRTOS integration packages as part of their firmware

offerings. We benchmark the two implementations using FreeRTOS v11.2 [30], running on the same two boards at 160 MHz with a 1 ms tick (time slice) per task.

4.3.1 Scenario design

We defined three scheduling scenarios that stress progressively more complex RTOS mechanisms:

1. **Basic processing** — a single counting thread runs at low priority alongside a high-priority reporting thread. The counting thread increments a global counter in a tight loop; the reporting thread wakes every 30 s and prints the counter delta. This isolates single-task throughput with minimal scheduler overhead.

2. **Cooperative processing** — five threads at the same low priority, each incrementing its own counter and then calling `taskYIELD()` to voluntarily yield the processor. A high-priority reporter prints all five counters every 30 s. This exercises voluntary context switching under equal-priority round-robin scheduling.

3. **Preemptive processing** — five threads at ascending priorities. Each thread resumes the next-higher-priority thread (which preempts it), increments its counter, then suspends itself. A high-priority reporter prints all five counters every 30 s. This exercises the full preemptive scheduler: suspend, resume, and priority-based preemption.

Table 4.9 summarises the three scenarios.

Table 4.9: FreeRTOS benchmark scenarios.

Scenario	**What it exercises**	**Metric**
Basic	Single-thread throughput, minimal scheduler overhead	Thread iterations/s
Cooperative	Voluntary context switching (`taskYIELD`)	Total thread iterations/30 s
Preemptive	Priority-based preemption, suspend/resume	Total thread iterations/30 s

4.3.2 Results

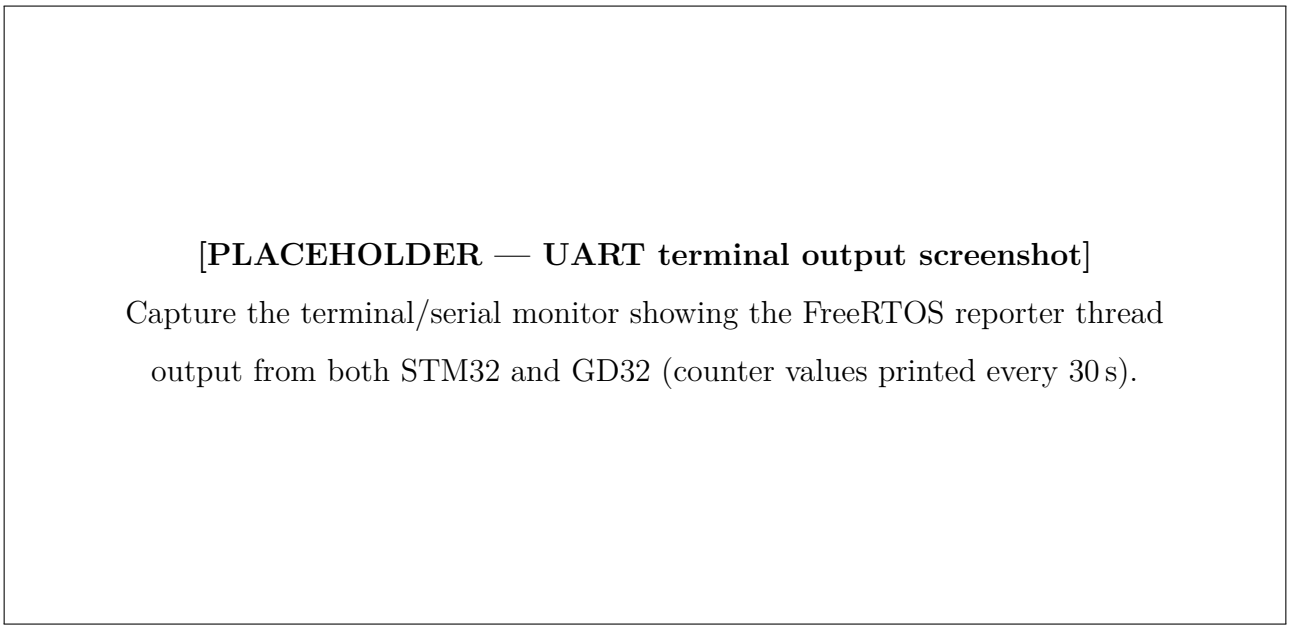

Figure 4.3: UART terminal output of the FreeRTOS cooperative benchmark: thread iteration counters reported every 30 seconds on STM32 (left) and GD32 (right).

4.3.2.1 Basic processing

Table 4.10 shows the performance result and Table 4.11 the footprint.

Table 4.10: FreeRTOS basic processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/s	47 943	47 955	−0.025 %

Table 4.11: FreeRTOS basic processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	14 266	10 128	+40.9 %
RAM	11 960	11 716	+2.1 %

Performance is virtually identical (−0.025 %), confirming that the counting loop is CPU-bound and both FreeRTOS ports schedule it with the same efficiency. The ROM difference (+40.9 %) is significant; the RAM difference is negligible.

4.3.2.2 Cooperative processing

Table 4.12: FreeRTOS cooperative processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/30 s	1 220 066	1 200 666	+1.6 %

Table 4.13: FreeRTOS cooperative processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	15 422	11 224	+37 %
RAM	10 972	10 728	+2.3 %

STM32 completes 1.6 % more thread iterations over 30 seconds, indicating a marginally faster `taskYIELD()` context switch. The footprint gap narrows slightly to +37 % ROM.

4.3.2.3 Preemptive processing

Table 4.14: FreeRTOS preemptive processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/30 s	294 615	285 144	+3.3 %

Table 4.15: FreeRTOS preemptive processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	16 226	12 028	+35 %
RAM	10 996	10 748	+2.3 %

Preemptive scheduling is the most demanding scenario, involving suspend, resume, and priority-based task switching. STM32 achieves 3.3 % more iterations, the largest performance advantage across the three FreeRTOS scenarios. The ROM gap (+35 %) is consistent with the other scenarios.

4.3.2.4 Per-layer ROM breakdown

To understand *where* the ROM overhead originates, Table 4.16 breaks down the ROM footprint into three architectural layers — application, HAL drivers, and FreeRTOS middleware stack — for all three scenarios.

Table 4.16: FreeRTOS ROM breakdown by layer (bytes).

Scenario	Layer	STM32U575	GD32E507	Diff.
Basic	Application	4 531	4 278	+6 %
	HAL drivers	4 927	1 078	+357 %
	MW stack	4 808	4 772	+0.7 %
Cooperative	Application	5 823	5 484	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	4 688	4 662	+0.6 %
Preemptive	Application	6 191	5 810	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	5 124	5 140	−0.3 %

The pattern is strikingly consistent across all three scenarios:

- The **FreeRTOS middleware stack** itself is virtually identical in size (≤1 % difference). Both vendors ship the same upstream FreeRTOS kernel, so this is expected.
- The **application layer** is ≈6 % larger on STM32, reflecting slightly more initialisation code in the STM32Cube project template.
- The **HAL layer** is 355–357 % larger on STM32, entirely accounting for the total ROM gap. The dominant contributor is the **HAL RCC** module, which configures the clock tree.

4.3.3 Interpretation

4.3.3.0.1 Performance. FreeRTOS scheduling performance is nearly identical between the two platforms in the basic scenario (−0.025 %) and slightly favours STM32 in the cooperative (+1.6 %) and preemptive (+3.3 %) scenarios. The advantage grows with scheduler complexity, suggesting that the

STM32Cube FreeRTOS port has a marginally more efficient context-switch implementation. However, these differences are small enough that application-level performance is unlikely to be affected.

4.3.3.0.2 ROM footprint. The ROM overhead follows the same pattern as the USB stack: a 35–41 % total increase driven almost entirely by the HAL layer, specifically the HAL RCC clock-configuration module. Crucially, the FreeRTOS middleware itself contributes no measurable overhead — the gap is in the vendor's hardware-abstraction code, not in the RTOS kernel.

4.3.3.0.3 RAM footprint. RAM differences are negligible (2.1–2.3 %), indicating that both FreeRTOS ports allocate task stacks and kernel structures similarly.

4.3.3.0.4 Enhancement proposals. The FreeRTOS results reinforce the conclusion from the USB benchmark: the HAL RCC module is the single largest contributor to STM32's ROM overhead across *all* middleware stacks. A targeted effort to slim the RCC driver — through dead-code elimination, conditional compilation based on the clocks actually used, or link-time optimisation — would benefit every STM32 project, not just FreeRTOS.

Conclusion

The ST-versus-GD32 comparison across two middleware stacks (USB device and FreeRTOS) and five scenarios reveals a consistent pattern:

- **Performance:** STM32 matches or slightly exceeds GD32 in every scenario. USB initialisation is 61.7 % faster; FreeRTOS preemptive scheduling is 3.3 % faster; steady-state throughput is equivalent.
- **ROM footprint:** STM32 consumes 35–88 % more flash, with the HAL layer (principally HAL RCC and, for USB, HAL/LL USB) responsible for the bulk of the difference. The middleware stacks themselves are comparable in size.
- **RAM footprint:** STM32 uses significantly less RAM than GD32 in USB scenarios (57–65 % less) thanks to smaller default stack/heap allocations and more static memory management. FreeRTOS RAM is nearly identical.
- **Architectural trade-off:** STM32Cube's modular, layered design (separate HAL, middleware, and user-configuration files) provides greater flexibility and reusability but at a measurable ROM

cost. GigaDevice's more monolithic approach yields smaller ROM by bundling driver and configuration logic into the middleware, at the expense of less user-visible modularity.

The single most impactful enhancement for STM32 would be **optimising the HAL RCC module**: it appears in every project and consistently accounts for the largest share of the HAL ROM gap. A secondary improvement for USB projects would be **finer-grained compile-time selection** of USB driver features (Full-Speed vs High-Speed, device vs OTG) to exclude unused code paths.

Chapter 5

Benchmarking ST against Texas Instruments

Introduction

Texas Instruments (TI) is one of the world's largest semiconductor manufacturers, with a broad portfolio of embedded processors and microcontrollers [31]. Its MSPM0 and MSPM3 families, built around Arm Cortex-M0+ and Cortex-M33 cores respectively, target cost-sensitive and mixed-signal applications with a strong emphasis on analog integration [32]. Unlike GigaDevice, which competes primarily on pin-compatibility, TI competes on a fundamentally different SDK architecture: a SysConfig-centric, unified serial peripheral model and a single-layer driver library (driverlib) rather than the dual HAL/LL approach used by STMicroelectronics.

This chapter presents the first phase of our ST-versus-TI comparison: a comprehensive **static analysis** of the low-level driver layers of both SDKs. Unlike the runtime-focused benchmarks in Chapter 4, this analysis examines source-code quality and SDK breadth without executing firmware on hardware. We compare the STM32CubeC5 Low-Layer (LL) drivers [39] against the TI MSPM33 SDK driverlib [34] across seven quality dimensions — lines of code, documentation coverage, code duplication, cyclomatic complexity, API surface design, MISRA C:2012 compliance, and security (CWE analysis). For the scope definition, we also map the SDK coverage in terms of peripheral drivers, middleware, cryptographic algorithms, and communication protocols, identifying what each vendor offers exclusively.

All measurements were produced by automated scripts using `cloc` [43], `cppcheck` [44] (with its `-addon=misra` and `-addon=cert` modes), `lizard` [45] for cyclomatic complexity, and `jscpd` [46] for duplication detection.

5.1 Hardware platforms

The analysis in this chapter targets driver code for two boards hosting comparable Cortex-M33 parts. Table 5.1 summarises their key characteristics.

Table 5.1: Board and MCU comparison — STM32C5A3 vs. MSPM33C321A.

Feature	**STM32C5A3 (NUCLEO-C5A3ZG)**	**MSPM33C321A (LP-MSPM33C321A)**
Core	Arm Cortex-M33 (FPU, DSP, MPU)	Arm Cortex-M33 (FPU, DSP, TrustZone)
Max frequency	144 MHz	160 MHz
Flash	1 MB (ECC, dual-bank)	1 MB (ECC, dual-bank, address swap)
SRAM	256 KB (128 KB with ECC)	256 KB (ECC)
ADC	3 × 12-bit, 4.5 MSPS	2 × 12-bit, 9.4 MSPS
DAC	1 × 12-bit	2 × 8-bit
CAN	2 × FDCAN	2 × CAN FD
USB	USB 2.0 FS (Host/Device/DRD)	—
Ethernet	Yes (MAC with DMA)	—
I3C	Yes	—
Debug probe	ST-LINK	XDS110 (EnergyTrace)

Both parts share the same core architecture and identical flash/RAM capacities, which makes the comparison fair at the silicon level. The STM32C5A3 offers broader connectivity (USB, Ethernet, I3C), while the MSPM33C321A favours higher analog bandwidth (9.4 MSPS ADC) and provides TrustZone support [33, 40].

5.2 Static analysis of the driver layer

This section compares the LL driver layer of STM32CubeC5 against the driverlib of the MSPM33 SDK. The LL layer was chosen for ST because it is the closest architectural equivalent to TI's driverlib: both are thin, inline-heavy abstractions over peripheral registers, designed for performance-critical code. We analyse seven complementary dimensions.

5.2.1 Scope and methodology

The analysis targets the following source sets:

- **ST:** all 33 header files in `stm32c5xx_drivers/ll/` (the LL layer of STM32CubeC5).
- **TI:** all 85 source and header files in `source/ti/driverlib/` and its `m33/` subdirectory (the driverlib of MSPM33 SDK v1.03.00.01).

Both sets were analysed on the same workstation under identical tool configurations. The tools and their roles are:

- **cloc** v2.08 [43] — counts blank, comment, and code lines per language and file.
- **lizard** [45] — computes cyclomatic complexity, number of parameters, and token count per function.
- **cppcheck** v2.x [44] — runs MISRA C:2012 addon (`-addon=misra`) and CWE/CERT checks.
- **jscpd** [46] — detects copy-pasted code blocks (duplication).
- A custom Python script (`analyze_firmware.py`) — extracts API surface metrics (public/static/inline functions, macros, enums, naming patterns, parameter distributions) and documentation coverage (Doxygen blocks, `@brief`, `@param`, `@return` tags, file headers).

5.2.2 Lines of code

Table 5.2 presents the line-count breakdown for each SDK's driver layer.

Table 5.2: Lines-of-code comparison — ST LL vs. TI driverlib.

Metric	ST (LL)	TI (driverlib)
Source files	33	85
Blank lines	5 744	8 747
Comment lines	57 649	45 763
Code lines	22 292	27 972
Total lines	85 685	82 482
Comment-to-code ratio	2.59	1.64

ST delivers its LL layer in 33 header-only files containing 22 292 lines of code, while TI spreads its driverlib across 85 files (both `.c` and `.h`) with 27 972 code lines. Despite having 25% more code, TI's total line count is slightly lower because ST invests significantly more in inline comments: 57 649

comment lines versus 45 763, yielding a comment-to-code ratio of 2.59 for ST compared with 1.64 for TI. This difference reflects ST's header-only, fully-documented inline function strategy, where each function carries extensive Doxygen annotations embedded alongside the implementation.

5.2.3 Documentation coverage

We measured documentation quality through Doxygen tag extraction. Table 5.3 summarises the results.

Table 5.3: Documentation coverage — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Total functions	3 713	3 056
Documented functions	3 613	2 525
Doc. coverage	**97.3 %**	**82.6 %**
Doxygen blocks	5 967	3 357
`@brief` tags	3 826	5 568
`@param` tags	4 440	3 970
`@return` tags	1 705	2 172
File headers	33	85
Inline comments	0	283

ST achieves 97.3% function-level documentation coverage, leaving only 100 undocumented functions. TI reaches 82.6%, with 531 undocumented functions — mostly in `.c` implementation files, where TI concentrates complex logic (e.g. flash controller operations, PLL configuration) without per-function Doxygen blocks. This architectural choice is deliberate: TI documents the *header* API thoroughly but omits Doxygen in `.c` bodies, whereas ST's header-only approach means every function definition is also its documented declaration.

Interestingly, TI uses more `@brief` tags (5 568 vs. 3 826) and more `@return` tags (2 172 vs. 1 705), suggesting that when TI does document a function, it tends to be descriptive about purpose and return values. ST, however, produces 78% more Doxygen blocks overall (5 967 vs. 3 357), reflecting its higher raw volume of documented entities.

5.2.4 Code duplication

Table 5.4 presents the duplication metrics.

Table 5.4: Code duplication — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Total lines analysed	85 685	82 403
Code lines	10 461	18 008
Duplicate blocks	71	414
Duplicate lines	229	1 306
Duplication %	**2.19 %**	**7.25 %**
Same-file clones	71	338
Cross-file clones	0	76

TI's driver layer exhibits 3.3× more duplication than ST's (7.25% vs. 2.19%). All 71 duplicate blocks in ST occur within the same file, typically in symmetric getter/setter pairs for multi-channel peripherals (e.g. ADC rank configuration). TI, by contrast, has 76 cross-file clones — code that is structurally identical across different peripheral modules. This pattern arises from TI's UniComm architecture, where `dl_unicommuart.h`, `dl_unicommspi.h`, and `dl_unicommi2cc.h` share a common General Serial Controller (GSC) base but replicate similar interrupt-handling and configuration boilerplate in each specialisation.

The higher duplication in TI is not inherently a quality defect — it reflects a design choice that trades some code reuse for peripheral-specific readability. However, from a maintenance perspective, ST's lower duplication means fewer locations to update when fixing a bug or adapting to silicon errata.

5.2.5 Cyclomatic complexity

Cyclomatic complexity (CC) measures the number of linearly independent paths through a function; higher values indicate more decision branches and, consequently, harder-to-test code [45]. We ran Lizard on every function in both driver sets and summarise the results in Table 5.5.

Table 5.5: Cyclomatic complexity comparison — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Analysed functions	3 542	2 539
Mean CC	4.44	7.35
Median CC	4	5
Maximum CC	49	141
Functions with CC $>$ 10	44 (1.2 %)	335 (13.2 %)
Functions with CC $>$ 20	12 (0.3 %)	97 (3.8 %)

ST's driver code is markedly flatter: the average function has fewer than five decision points, and only 1.2 % of functions exceed the widely accepted threshold of CC = 10 [47]. The maximum (CC = 49) occurs in a temperature-calculation helper (`LL_ADC_CALC_TEMPERATURE`) that contains a long chain of calibration look-ups.

TI's SDK shows a heavier tail: 13.2 % of its functions exceed CC = 10, with a peak of 141 in a single function. Many of these high-complexity functions reside in the UniComm serial-controller layer, where a large `switch` dispatches configuration variants for UART, SPI, and I^2C through a shared code path. While this design consolidates peripheral logic, it concentrates branching in ways that complicate unit testing and formal verification.

From a maintainability standpoint, ST's approach of decomposing peripheral logic into many small, low-complexity inline functions aligns better with safety-coding guidelines that recommend CC $\leq$ 10 per function.

5.2.6 API surface

Table 5.6 compares the API design characteristics.

Table 5.6: API surface comparison — ST LL vs. TI driverlib.

Metric	**ST (LL)**	**TI (driverlib)**
Public functions	3 543	2 500
Static functions	1	42
Inline functions	3 543	2 032
Functional macros	194	27
Constant macros	3 993	3 271
Enums	1	361
Typedefs	0	470
Avg. parameters/func.	1.17	1.69
Max parameters	8	8
`extern "C"` guards	33	43
Include guards	33	0
Parameter distribution (functions by param count)		
0 parameters	492	120
1 parameter	2 303	1 160
2 parameters	658	933
3 parameters	135	274
4+ parameters	48	139

The two APIs reflect fundamentally different design philosophies:

- **ST LL** exposes 3 543 public functions, *all* of which are `__STATIC_INLINE`. The API is almost entirely composed of one-parameter getters and zero-parameter status queries (65% of functions take 0 or 1 parameter). Type safety relies on constant macros rather than enums. The naming convention follows the `LL_PERIPHERAL_Action` pattern consistently across all 33 files.

- **TI driverlib** exposes 2 500 public functions, of which 2 032 are inline (81%). It makes heavier use of enums (361 enum types) and typedefs (470) for configuration parameters, which improves type safety and IDE autocompletion at the cost of a more complex type graph. The naming convention follows `DL_Peripheral_Action` uniformly. Functions tend to take more parameters on average (1.69 vs. 1.17), reflecting a more configuration-object-oriented style where a single call sets multiple related fields.

ST's 194 functional macros versus TI's 27 show that ST still uses preprocessor computation in places (e.g. register-offset calculations, bit-field assembly) where TI has migrated the same logic into inline functions. While both approaches compile to equivalent machine code, the inline-function approach provides better debugging and type checking.

5.2.7 MISRA C:2012 compliance

Both SDKs were analysed using `cppcheck` with the MISRA C:2012 addon [47]. Table 5.7 presents the per-rule breakdown.

Table 5.7: MISRA C:2012 violations — ST LL vs. TI driverlib.

Rule	Description	ST	TI
17.3	Function declared implicitly	3 541	281
11.4	Conversion between pointer and integer	84	4
10.6	Composite expression assigned to wider type	29	—
20.7	Macro parameter not enclosed in parentheses	21	—
7.3	Lower-case "l" suffix on literal	18	—
10.4	Incompatible essential types in operation	—	67
12.1	Precedence of operators unclear	—	52
18.4	Pointer arithmetic	—	76
15.5	Multiple return statements	4	14
10.1	Operands of inappropriate essential type	—	13
3.1	Comment contains nested comment	—	11
Other	Various minor rules	6	33
Total		**3 703**	**551**

Interpretation

At first glance, ST's LL layer appears to have far more MISRA violations (3 703 vs. 551). However, this requires careful interpretation:

1. **Rule 17.3 dominates ST's count.** Of ST's 3 703 violations, 3 541 (96%) are Rule 17.3 (implicit function declaration). These arise because every LL function is defined as `__STATIC_INLINE` in a header and `cppcheck` analyses each header in isolation, reporting each inline function call within

the same header as an implicit declaration when the callee is defined later in the file. In a real compilation unit (where the header is `#included`), these are all resolved. This is a *tooling artifact*, not a genuine code-quality issue.

2. **Rule 11.4 is unavoidable in MCU drivers.** The 84 Rule 11.4 violations in ST are casts between integers and peripheral base-address pointers (`(ADC_TypeDef *)ADC1_BASE`). Both vendors do this; TI has the same pattern but fewer instances (4) because TI defines peripheral pointers via device-header macros that `cppcheck` does not fully expand.

3. **TI's violations are more diverse.** TI triggers 15 distinct MISRA rules compared with ST's 9. Rules 18.4 (pointer arithmetic, 76 occurrences), 10.4 (essential-type mismatch, 67), and 12.1 (operator precedence, 52) point to more complex expression patterns in TI's `.c` implementation files.

If we exclude the tooling-artifact Rule 17.3 from both vendors, the adjusted counts are ST = 162 and TI = 270, putting ST ahead on genuine MISRA conformance per line of code.

Security analysis (CWE)

Both driver layers returned **zero CWE findings** from `cppcheck`'s CERT/CWE checker. Neither SDK contains patterns flagged as common weakness enumerations, which is expected for thin, register-level driver code that performs no dynamic allocation, string handling, or network I/O.

5.3 SDK coverage comparison

Beyond driver-layer quality, the breadth of an SDK — how many peripherals, middleware stacks, and communication protocols it supports out of the box — determines the development effort required for a real product. We mapped the complete contents of both SDKs file by file and identified exclusive features on each side.

5.3.1 Peripheral driver coverage

Table 5.8 lists the peripheral drivers present in only one SDK.[1]

[1]For brevity, only a representative subset of ST's 25 exclusive drivers is shown; the full list is available in the analysis output.

Table 5.8: Exclusive peripheral drivers — features in one SDK only.

Vendor	Driver	Capability
ST-exclusive (25 drivers)		
ST	CORDIC accelerator	HW trigonometric/hyperbolic math
ST	DAC (12-bit)	Full DAC peripheral
ST	Ethernet MAC	On-chip networking
ST	USB Host / Device / DRD	Full USB stack support
ST	I3C controller	Next-gen I3C bus
ST	HASH accelerator (SHA)	HW SHA-1/SHA-256
ST	OPAMP	On-chip operational amplifier
ST	ICACHE controller	Configurable instruction cache
ST	XSPI (Octo/Quad-SPI)	Advanced memory-mapped SPI
ST	LPUART, USART, SmartCard, SMBus	Extended serial protocols
TI-exclusive (16 drivers)		
TI	High-Speed ADC (HSADC)	Dedicated 9.4 MSPS ADC block
TI	General Serial Controller (GSC)	Unified serial engine
TI	UniComm (I2C/SPI/UART)	Configurable serial block
TI	Signal Processing Subsystem	On-chip analog processing
TI	Low-Frequency Subsystem (LFSS)	Unified RTC + tamper + scratchpad
TI	Key Store Controller	HW key management
TI	Error Action Module	HW automatic fault handling
TI	UART extend (IrDA, Manchester)	Extended UART modes

ST leads with 25 exclusive peripheral drivers to TI's 16. The most significant gaps are in connectivity: ST offers USB, Ethernet, and I3C, which are entirely absent from the MSPM33C321A. TI compensates with stronger analog integration (high-speed ADC, signal processing subsystem) and a unified serial architecture that consolidates UART, SPI, and I2C into a single configurable peripheral.

5.3.2 Middleware coverage

Table 5.9 compares the middleware stacks.

Table 5.9: Exclusive middleware — present in one SDK only.

Vendor	Middleware	Purpose
ST	FileX	FAT-compatible embedded file system
ST	LevelX	Flash wear-leveling (NAND/NOR)
ST	LwIP	Lightweight TCP/IP stack
ST	USBX	USB device/host class drivers
ST	mbedTLS	TLS/SSL library (TLS 1.2/1.3)
ST	Open Bootloader	Multi-interface bootloader
ST	stFCF	Security provisioning framework
ST	stCryptoLib	Comprehensive crypto library
TI	Edge AI	On-device ML inference
TI	LVGL	Embedded graphics/GUI library
TI	LIN Bus	Automotive LIN protocol
TI	DALI	Smart lighting protocol
TI	IQmath	Fixed-point DSP math library
TI	Post-Quantum Crypto	ML-DSA (Dilithium) signatures
TI	Curve25519	Modern elliptic curve crypto

ST provides 8 exclusive middleware components focused on connectivity (USB, Ethernet/IP, file system) and security (TLS, crypto library, secure boot). TI provides 7 exclusive components that target emerging domains: Edge AI for on-device machine learning, LVGL for embedded GUIs, automotive protocols (LIN, DALI), and forward-looking cryptography (post-quantum ML-DSA, Curve25519).

5.3.3 Cryptographic algorithm coverage

Table 5.10 provides a detailed comparison of the algorithms available in each vendor's crypto libraries.

Table 5.10: Cryptographic algorithm coverage comparison.

Algorithm	**ST (stCryptoLib)**	**TI (crypto/)**
AES (ECB, CBC, CTR, GCM)	✓	✓
AES-CCM / AES-XTS	✓	—
SHA-224/256	✓	✓
SHA-384/512, SHA-3	✓	—
HMAC	✓	✓
CMAC / KMAC	✓	—
RSA (PKCS#1)	✓	✓
ECDSA / ECDH	✓	✓
EdDSA (Ed25519)	✓	✓
SM2 / SM3 (Chinese std.)	✓	—
ChaCha20-Poly1305	✓	—
ML-DSA (Post-Quantum)	—	✓
Curve25519 / X25519	—	✓
TRNG (HW)	✓	✓
PKA (HW)	✓	✓
Key Store (HW)	✓	✓

ST offers the broader algorithm portfolio (10+ exclusive algorithms), covering legacy standards (SHA-1), Chinese national standards (SM2/SM3), and authenticated-encryption modes (AES-CCM, ChaCha20-Poly1305). TI, while narrower, holds a forward-looking advantage with post-quantum cryptography (ML-DSA/Dilithium) and modern elliptic-curve primitives (Curve25519), which positions it well for long-term security requirements [48].

5.3.4 Communication protocol coverage

Table 5.11 summarises the communication protocol support.

Table 5.11: Communication protocol support comparison.

Protocol	**ST**	**TI**
UART / SPI / I2C / I2S / CAN FD	✓	✓
USB (Device + Host)	✓	—
Ethernet	✓	—
I3C	✓	—
SMBus / SmartCard (ISO 7816)	✓	—
LPUART / USART (synchronous)	✓	—
IrDA mode / Manchester encoding	—	✓
LIN Bus	—	✓

ST supports 7 exclusive communication protocols, with USB and Ethernet being the most impactful for connected applications. TI brings 3 exclusive capabilities: IrDA mode, Manchester encoding (both through its extended UART), and LIN Bus for automotive applications.

5.4 Summary of findings

Table 5.12 provides a consolidated view of the static-analysis results.

Table 5.12: Static-analysis scoreboard — STM32CubeC5 LL vs. TI MSPM33 driverlib.

Dimension	**ST result**	**TI result**	**Advantage**
Code lines	22 292	27 972	ST (leaner)
Documentation coverage	97.3 %	82.6 %	ST
Code duplication	2.19 %	7.25 %	ST
Cyclomatic complexity	4.44 avg	7.35 avg	ST (flatter)
API functions	3 543	2 500	ST (wider)
Type safety (enums)	1	361	TI
MISRA violations	162*	270	ST
Security (CWE)	0	0	Tie
Exclusive peripherals	25	16	ST
Exclusive middleware	8	7	Balanced
Crypto algorithms	10+ exclusive	2 exclusive	ST (breadth), TI (PQC)
Comm. protocols	7 exclusive	3 exclusive	ST

*After excluding tooling-artifact Rule 17.3 from both vendors.

The static analysis reveals that ST's LL driver layer is leaner, better documented, less duplicated, and more MISRA-conformant when tooling artifacts are factored out. ST also holds a clear lead in SDK breadth, particularly in connectivity (USB, Ethernet, I3C) and security middleware (TLS, comprehensive crypto library).

TI, however, brings distinct strengths: stronger type safety through enums and typedefs, forward-looking post-quantum cryptography, superior analog integration (9.4 MSPS ADC), a unified serial architecture that simplifies board design, and emerging-domain middleware (Edge AI, LVGL, automotive protocols). These advantages position TI well for mixed-signal and AI-at-the-edge applications where connectivity is less critical than analog performance.

Conclusion

This chapter presented the first phase of our ST-versus-TI comparison, focusing on a comprehensive static analysis of the low-level driver layers of STM32CubeC5 and the TI MSPM33 SDK. The analysis covered seven quality dimensions and a complete SDK coverage mapping.

The results demonstrate that ST's ecosystem is the more mature and broadly equipped platform, excelling in documentation discipline, code hygiene, MISRA conformance, and middleware breadth. TI offers a credible alternative with targeted advantages in analog performance, type-safe API design, forward-looking cryptography, and emerging middleware for AI and automotive applications.

Future work will extend this comparison to runtime benchmarks — footprint analysis, execution time, and power consumption on identical workloads — to complement the static-analysis findings presented here with empirical measurements from both platforms.

General Conclusion and Perspectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

Nullam fermentum id mauris suscipit varius. Quisque tristique tempor fringilla. Nam porta tincidunt orci in consectetur. Suspendisse nec sem nisi. Suspendisse potenti. Sed sodales aliquam libero, at dapibus ligula accumsan non. Quisque congue et urna a consectetur. Nullam maximus venenatis ornare. Donec in luctus urna, vel rhoncus lectus. Etiam lobortis, lacus nec gravida iaculis, magna nulla iaculis leo, quis congue nunc tortor pellentesque lorem. Sed fermentum vulputate dui, ac malesuada eros molestie sit amet. Proin fringilla elit justo, in posuere urna porttitor et. Curabitur dictum justo vitae metus pellentesque, eget feugiat sem feugiat.

Curabitur in mauris eu felis cursus auctor eget ut massa. Curabitur eleifend consectetur magna in ultrices. Etiam ut semper turpis. Morbi sed ipsum nec sem rutrum blandit sit amet vitae felis. Vestibulum vitae hendrerit diam. Aliquam pellentesque est mi, et tempus nisl laoreet in. Maecenas consequat augue a ante congue dignissim. In condimentum erat in volutpat tempus. Mauris vulputate, enim ut dignissim varius, lorem purus tristique sapien, lobortis accumsan dolor lorem quis eros. Duis vel imperdiet metus, non suscipit tortor.

Appendix A

Glossary

Architectural layer: A classification of firmware symbols into one of four categories — application, middleware, low-level driver, or system runtime — used to attribute memory costs to comparable functional blocks across vendors.

Board Support Package (BSP): A set of drivers and configuration files that adapt a vendor's generic HAL to a specific evaluation or development board.

Context switch: The process by which an RTOS saves the CPU state of the currently running task and restores the state of the next task to be executed. Context-switch efficiency directly affects multi-threaded throughput.

Cross-vendor benchmark: A controlled experiment in which the same workload runs on two vendor platforms under identical conditions (clock, optimisation level, scenario logic), producing comparable metrics.

Firmware footprint: The amount of non-volatile (flash/ROM) and volatile (RAM) memory a firmware image occupies once compiled and linked. It is a static measurement taken from the linked binary, not a dynamic runtime observation.

Linker map: A text file produced by the linker at the end of the build process. It lists every module and function linked into the final image together with its code size, constant-data size, and variable-data size. It is the primary input to the footprint analysis tool.

Middleware: Software libraries that sit between the hardware abstraction layer and the application, providing protocol or service logic (e.g. USB device stack, FreeRTOS kernel, FatFS, LwIP).

Semantic equivalence: The property that two benchmark implementations — one on the reference platform and one on the competitor — exhibit identical observable behaviour (task structure, timing, synchronisation, reporting format) so that their comparison is fair.

Software stack: A vertical slice of firmware functionality (e.g. USB device, FreeRTOS integration) comprising application code, middleware, and hardware drivers, benchmarked as a unit.

Tick (RTOS): The periodic timer interrupt that drives the RTOS scheduler. A 1 ms tick means the scheduler evaluates task readiness every millisecond.

Appendix B

Acronyms

AES	Advanced Encryption Standard
AI	Artificial Intelligence
API	Application Programming Interface
BSP	Board Support Package
CAN	Controller Area Network
CDC	Communication Device Class
CLI	Command-Line Interface
CMSIS	Cortex Microcontroller Software Interface Standard
CRC	Cyclic Redundancy Check
CPU	Central Processing Unit
CSV	Comma-Separated Values
CWE	Common Weakness Enumeration
DAC	Digital-to-Analog Converter
DMA	Direct Memory Access
DRD	Dual-Role Device (USB)
DSP	Digital Signal Processing
DWT	Data Watchpoint and Trace
ECC	Error Correction Code
ECDSA	Elliptic Curve Digital Signature Algorithm
EWARM	Embedded Workbench for ARM
FDCAN	Flexible Data-rate CAN
FPU	Floating-Point Unit
FS	Full-Speed (USB)
FSP	Flexible Software Package (Renesas)
FST	Faculty of Sciences of Tunis

LFSS Low-Frequency Subsystem

LIN Local Interconnect Network

LL Low-Layer (driver)

LLM Large Language Model

LOC Lines of Code

MB Megabyte

MCD Microcontrollers Division

MCU Microcontroller Unit

MISRA Motor Industry Software Reliability Association

ML-DSA Module-Lattice Digital Signature Algorithm

MPU Memory Protection Unit

MSC Mass Storage Class

MSPS Mega Samples Per Second

MW Middleware

NXP NXP Semiconductors

OTG On-The-Go (USB)

PDF Portable Document Format

PFE Projet de Fin d'Études

PKA Public Key Accelerator

RAM Random-Access Memory

RCC Reset and Clock Control

ROM Read-Only Memory (flash)

RTOS Real-Time Operating System

SDK Software Development Kit

SQL Structured Query Language

SRAM Static Random-Access Memory

ST STMicroelectronics

SWO Serial Wire Output

TI Texas Instruments

TLS Transport Layer Security

TRNG True Random Number Generator

UART Universal Asynchronous Receiver-Transmitter

USB Universal Serial Bus

UTM University of Tunis El Manar

VS Code Visual Studio Code

Appendix C

Complementary Code Listings

This appendix provides representative code excerpts that complement the tool and benchmark descriptions in the main chapters. They illustrate the formats consumed and produced by our tools and the firmware structure of the benchmark scenarios.

C.1 IAR EWARM linker map excerpt

Listing C.1 shows an abbreviated IAR EWARM linker map for a USB HID project. This is the format parsed by MCU Workflow Studio (Chapter 2). Each entry lists a module, its read-only code size, read-only data, and read-write data — the three quantities from which flash and RAM footprint are derived.

```
*******************************************************************************
*** MODULE SUMMARY
***

    Module                ro code  ro data  rw data
    ------                -------  -------  -------
    main.o                    164       --       --
    stm32u5xx_hal.o           212       --        4
    stm32u5xx_hal_gpio.o      456       --       --
    stm32u5xx_hal_rcc.o     4 560       16       --
    stm32u5xx_hal_pcd.o     2 496       --       --
    stm32u5xx_ll_usb.o      2 492       --       --
    usbd_core.o             1 048       --       --
    usbd_ctlreq.o             876       --       --
    usbd_hid.o                620       --       --
```

```
    usbd_conf.o               524        --        32
    usbd_desc.o               461       268        --
    system_stm32u5xx.o        128        16         4
    startup_stm32u575xx.o     394       512        --
    ------                 -------   -------   -------
    Total:                 14 431       812        40

*******************************************************************************
*** ENTRY LIST
***

    Entry                  Address    Size  Type  Object
    -----                  -------    ----  ----  ------
    HAL_Init               0x0800'1234   52  Code  stm32u5xx_hal.o
    HAL_RCC_OscConfig      0x0800'2000  1024  Code  stm32u5xx_hal_rcc.o
    HAL_RCC_ClockConfig    0x0800'2400   896  Code  stm32u5xx_hal_rcc.o
    USBD_Init              0x0800'3000   128  Code  usbd_core.o
    USBD_HID_Init          0x0800'3200    96  Code  usbd_hid.o
    ...
```

Listing C.1: Abbreviated IAR EWARM linker map (USB HID, STM32U575).

C.2 Agent configuration file

Listing C.2 shows the configuration of the Creation-Porting specialist agent from the MCU Benchmark Copilot Agent system (Chapter 3). The YAML frontmatter declares metadata, description, argument hints, and tool permissions; the Markdown body contains the behavioural instructions, autonomy contract, and supported workflow directions.

```
---
name: Creation-Porting
description: "Create benchmark projects from scratch or port existing
  benchmarks in any direction (STM32->competitor, competitor->STM32,
  vendor->vendor, board->board). Use when: generating target project
  files, creating from a scenario description, separating
  portable/RTOS/board logic, or acquiring missing reference firmware."
argument-hint: A benchmark requirement, scenario description, scenario
  name, firmware name, or a reference project plus target platform.
tools: ['read', 'search', 'edit', 'vscode', 'todo', 'web', 'execute']
---

# Creation-Porting Agent

## Supported Directions
- From scratch (scenario description only)
- STM32 -> competitor
- Competitor -> STM32
- Vendor A -> Vendor B
- Board A -> Board B
- Name only (resolve, acquire, create)

## Autonomy Contract
- Operates autonomously for file generation, project creation, and
  material acquisition (web fetches from official sources).
- Asks only before flashing/programming/resetting hardware.
- Acquires missing firmware, CMSIS, BSP, startup, linker from
  official vendor GitHub repos and SDKs without confirmation.

## IAR Template Resolution
Before creating any project:
1. Look up target board in config/iar_templates.yaml
```

```
2. Follow resolution: exact board -> family fallback -> type fallback
3. Acquire template if status == "needs_download"
4. Use materialised template as read-only project baseline
5. Place benchmark code through app_integration_points only

## Multi-Layer Separation
1. Portable application layer (writable) - benchmark logic
2. RTOS / middleware layer (protected) - select, don't modify
3. Board-specific layer (protected) - select existing vendor files
4. Build surface (configure only) - .ewp references, defines, paths

## Semantic Preservation Rule
Preserve semantics and observable behaviors, not vendor API names.
Map CMSIS-RTOS2 <-> native FreeRTOS <-> ThreadX as needed via
app-level adapter files. Never modify vendor HAL source.
```

Listing C.2: Creation-Porting agent configuration (abbreviated).

C.3 FreeRTOS cooperative benchmark scenario

Listing C.3 shows the core task logic of the cooperative processing benchmark scenario from Chapter 4. Five equal-priority threads each increment a counter and yield; a reporter thread prints the results every 30 seconds.

```
/* Shared counters for the five worker threads */
volatile uint32_t counters[5] = {0};

/* Worker task: increment own counter, then yield */
void WorkerTask(void *param)
{
    uint32_t id = (uint32_t)param;
    for (;;)
    {
        counters[id]++;
        taskYIELD();
    }
}

/* Reporter task: print all counters every 30 s */
void ReporterTask(void *param)
{
    (void)param;
    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(30000));
        printf("Counters: %lu %lu %lu %lu %lu\r\n",
               counters[0], counters[1], counters[2],
               counters[3], counters[4]);
        /* Reset for next interval */
        for (int i = 0; i < 5; i++)
            counters[i] = 0;
    }
}

/* Task creation (in main, after HAL and kernel init) */
void CreateBenchmarkTasks(void)
{
```

```
34      for (uint32_t i = 0; i < 5; i++)
35      {
36          xTaskCreate(WorkerTask, "Worker", 128,
37                      (void *)i, tskIDLE_PRIORITY + 1, NULL);
38      }
39      xTaskCreate(ReporterTask, "Reporter", 256,
40                  NULL, tskIDLE_PRIORITY + 2, NULL);
41      vTaskStartScheduler();
42  }
```

Listing C.3: FreeRTOS cooperative scenario — task functions (STM32, simplified).

C.4 Footprint tool CLI output

Listing C.4 shows a representative terminal output from a footprint benchmark run using the MCU Workflow Studio command-line interface (Chapter 2).

```
$ mcu-workflow-cli benchmark \
    --source stm32_usb_hid.map --source-vendor stm32 \
    --target gd32_usb_hid.map  --target-vendor gd32 \
    --out results/

[INFO] Parsing source map: stm32_usb_hid.map (IAR EWARM)
[INFO] Parsing target map: gd32_usb_hid.map (IAR EWARM)
[INFO] Classifying 47 source symbols into layers...
[INFO] Classifying 31 target symbols into layers...
[INFO] Mapping engine: 42 pairs scored, 38 accepted (confidence >= 0.72)
[INFO] 4 pairs queued for review (confidence 0.55-0.72)

=== FOOTPRINT COMPARISON (USB HID) ===

Layer           | STM32 ROM | GD32 ROM  | Diff
----------------|-----------|-----------|--------
Application     |   2 580 B |   1 942 B | +33%
HAL drivers     |  10 426 B |     820 B | +1171%
MW stack        |   2 859 B |   5 743 B | -50%
----------------|-----------|-----------|--------
TOTAL ROM       |  16 269 B |   9 175 B | +77%
TOTAL RAM       |   3 446 B |   9 608 B | -65%

Verdict: STM32 uses +77% more ROM but -65% less RAM.
Root cause: HAL RCC (4560 B) + HAL/LL USB (4988 B) = 93% of gap.

[INFO] Results written to results/benchmark_results.xlsx
[INFO] Mapping diagnostics: results/mapping_diagnostics.json
[INFO] Review queue (4 items): results/review_queue.csv
```

Listing C.4: MCU Workflow Studio CLI — benchmark summary output.

C.5 Static-analysis automation script (ST vs. TI)

Listing C.5 shows an abbreviated version of the Python script used to extract API surface and documentation coverage metrics for the ST-versus-TI static analysis (Chapter 5). The full script analyses duplication, documentation, and API design in a single pass over the driver source trees.

```
"""
Firmware benchmark analysis script:
- Code duplication detection (sliding-window hash)
- Documentation coverage (Doxygen tag extraction)
- API surface area analysis (functions, macros, enums)
"""
import os, re, json, hashlib
from collections import defaultdict, Counter
from pathlib import Path

TI_DIR = r"mspm33_sdk/source/ti/driverlib"
ST_DIR = r"STM32CubeC5/stm32c5xx_drivers/ll"

def get_c_files(directory):
    files = []
    for root, _, filenames in os.walk(directory):
        for f in filenames:
            if f.endswith(('.c', '.h')):
                files.append(os.path.join(root, f))
    return files

def analyze_api(files):
    """Extract API surface metrics from C source files."""
    result = {
        'public_functions': 0, 'static_functions': 0,
        'inline_functions': 0, 'macros_functional': 0,
        'macros_constant': 0, 'enums': 0, 'typedefs': 0,
        'param_distribution': Counter(),
    }
    for filepath in files:
        with open(filepath, 'r', errors='ignore') as f:
            content = f.read()
        # Count public functions (not static)
```

```
        pub = re.findall(
            r'(?<!static\s)(?:__STATIC_INLINE\s+|inline\s+)?'
            r'(?:void|bool|uint\w+|int\w+|DL_\w+|LL_\w+)\s+'
            r'(\w+)\s*\(([^)]*)\)', content)
        for name, params in pub:
            result['public_functions'] += 1
            n = len([p for p in params.split(',')
                     if p.strip()]) if params.strip() else 0
            result['param_distribution'][n] += 1
        # Count enums, typedefs, macros
        result['enums'] += len(re.findall(
            r'\benum\s+\w*\s*\{', content))
        result['typedefs'] += len(re.findall(
            r'\btypedef\s+', content))
        result['macros_functional'] += len(re.findall(
            r'#define\s+\w+\([^)]*\)', content))
        result['macros_constant'] += len(re.findall(
            r'#define\s+\w+\s+[^(]', content))
    return result

if __name__ == '__main__':
    for label, path in [("TI", TI_DIR), ("ST", ST_DIR)]:
        files = get_c_files(path)
        api = analyze_api(files)
        print(f"\n=== {label} API Surface ===")
        for k, v in api.items():
            print(f"  {k}: {v}")
```

Listing C.5: Abbreviated `analyze_firmware.py` — API and documentation extraction.

Bibliography

[1] STMicroelectronics, *About ST*. [Online]. Available: https://www.st.com/content/st_com/en/about/st_company_information.html. Accessed: 2026.

[2] STMicroelectronics, *2024 Annual Report and Financial Highlights*. [Online]. Available: https://investors.st.com/. Accessed: 2026.

[3] STMicroelectronics, *Strategy and Innovation*. [Online]. Available: https://www.st.com/content/st_com/en/about/innovation---technology/innovation---technology.html. Accessed: 2026.

[4] STMicroelectronics, *STM32 32-bit Arm Cortex MCUs*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html. Accessed: 2026.

[5] Arm Ltd., *Cortex-M Series Processors*. [Online]. Available: https://developer.arm.com/Processors/Cortex-M. Accessed: 2026.

[6] STMicroelectronics, *STM32Cube Ecosystem*. [Online]. Available: https://www.st.com/en/ecosystems/stm32cube.html. Accessed: 2026.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[8] Python Software Foundation, *The Python Language Reference, version 3.11*. [Online]. Available: https://docs.python.org/3.11/. Accessed: 2026.

[9] The Qt Company, *Qt for Python (PySide6) Documentation*. [Online]. Available: https://doc.qt.io/qtforpython/. Accessed: 2026.

[10] The pandas development team, *pandas: Powerful Python Data Analysis Toolkit*. [Online]. Available: https://pandas.pydata.org/. Accessed: 2026.

[11] E. Gazoni and C. Clark, *openpyxl: A Python Library to Read and Write Excel 2010 xlsx/xlsm Files*. [Online]. Available: https://openpyxl.readthedocs.io/. Accessed: 2026.

[12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/.

[13] SQLite Development Team, *SQLite Documentation*. [Online]. Available: https://www.sqlite.org/docs.html. Accessed: 2026.

[14] PyInstaller Development Team, *PyInstaller Manual*. [Online]. Available: https://pyinstaller.org/. Accessed: 2026.

[15] IAR Systems, *IAR Embedded Workbench for Arm — C/C++ Development Guide (ILINK Linker and Linker Map File)*. [Online]. Available: https://www.iar.com/embedded-development-tools/iar-embedded-workbench. Accessed: 2026.

[16] GitHub, *GitHub Copilot Documentation*. [Online]. Available: https://docs.github.com/en/copilot. Accessed: 2026.

[17] Anthropic, *Claude API Documentation*. [Online]. Available: https://docs.anthropic.com/. Accessed: 2026.

[18] OpenAI, *OpenAI API Reference*. [Online]. Available: https://platform.openai.com/docs/. Accessed: 2026.

[19] Ollama, *Ollama: Run Large Language Models Locally*. [Online]. Available: https://ollama.com/. Accessed: 2026.

[20] GitHub, *GitHub Models Documentation*. [Online]. Available: https://docs.github.com/en/github-models. Accessed: 2026.

[21] NXP Semiconductors, *MCUXpresso SDK Documentation*. [Online]. Available: https://mcuxpresso.nxp.com/. Accessed: 2026.

[22] Microsoft, *Visual Studio Code Documentation*. [Online]. Available: https://code.visualstudio.com/docs. Accessed: 2026.

[23] Microsoft, *GitHub Copilot Agent Mode in VS Code*. [Online]. Available: https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode. Accessed: 2026.

[24] Microsoft, *Custom Chat Agents and Skills — VS Code Documentation*. [Online]. Available: https://code.visualstudio.com/docs/copilot/copilot-customization. Accessed: 2026.

[25] IAR Systems, *C-SPY Debugging Guide — Batch Mode (cspybat)*. [Online]. Available: https://www.iar.com/knowledge/support/technical-notes/debugger/. Accessed: 2026.

[26] GigaDevice Semiconductor, *GD32 Microcontrollers*. [Online]. Available: https://www.gigadevice.com/microcontroller. Accessed: 2026.

[27] GigaDevice Semiconductor, *GD32E507xx Datasheet*, Rev. 1.2, 2022. [Online]. Available: https://www.gigadevice.com/microcontroller/gd32e507zet6. Accessed: 2026.

[28] STMicroelectronics, *STM32U575/585 Datasheet — Ultra-low-power with FPU Arm Cortex-M33 MCU*. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32u575zi.pdf. Accessed: 2026.

[29] Amazon Web Services, *FreeRTOS — Real-Time Operating System for Microcontrollers*. [Online]. Available: https://www.freertos.org/. Accessed: 2026.

[30] Amazon Web Services, *FreeRTOS V11.2.0 Release Notes*. [Online]. Available: https://github.com/FreeRTOS/FreeRTOS-Kernel/releases/tag/V11.2.0. Accessed: 2026.

[31] Texas Instruments, *About TI*. [Online]. Available: https://www.ti.com/about-ti/company/company-overview.html. Accessed: 2026.

[32] Texas Instruments, *MSPM33C321A Product Page*. [Online]. Available: https://www.ti.com/product/MSPM33C321A. Accessed: 2026.

[33] Texas Instruments, *MSPM33C321A Datasheet*. [Online]. Available: https://www.ti.com/document-viewer/MSPM33C321A/datasheet. Accessed: 2026.

[34] Texas Instruments, *MSPM33-SDK — Software Development Kit for MSPM33*. [Online]. Available: https://www.ti.com/tool/MSPM33-SDK. Accessed: 2026.

[35] Texas Instruments, *LP-MSPM33C321A LaunchPad Development Kit*. [Online]. Available: https://www.ti.com/tool/LP-MSPM33C321A. Accessed: 2026.

[36] Texas Instruments, *SysConfig — System Configuration Tool*. [Online]. Available: https://www.ti.com/tool/SYSCONFIG. Accessed: 2026.

[37] Texas Instruments, *Code Composer Studio IDE*. [Online]. Available: https://www.ti.com/tool/CCSTUDIO. Accessed: 2026.

[38] Texas Instruments, *TI Arm Clang Compiler Tools*. [Online]. Available: https://www.ti.com/tool/ARM-CGT. Accessed: 2026.

[39] STMicroelectronics, *STM32CubeC5 — STM32Cube MCU Package for STM32C5 Series*. [Online]. Available: https://www.st.com/en/embedded-software/stm32cubec5.html. Accessed: 2026.

[40] STMicroelectronics, *STM32C5A3ZG Product Page*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32c5a3zg.html. Accessed: 2026.

[41] STMicroelectronics, *NUCLEO-C5A3ZG Evaluation Board*. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-c5a3zg.html. Accessed: 2026.

[42] STMicroelectronics, *STM32C5 Series Overview*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32c5-series.html. Accessed: 2026.

[43] A. Danial, *cloc — Count Lines of Code*, version 2.08. [Online]. Available: https://github.com/AlDanial/cloc. Accessed: 2026.

[44] D. Marjamäki *et al.*, *Cppcheck — A Static Analysis Tool for C/C++ Code*. [Online]. Available: https://cppcheck.sourceforge.io/. Accessed: 2026.

[45] T. Yin, *Lizard — A Code Complexity Analyser*. [Online]. Available: https://github.com/terryyin/lizard. Accessed: 2026.

[46] K. Kravets, *jscpd — Copy/Paste Detector for Source Code*. [Online]. Available: https://github.com/kucherenko/jscpd. Accessed: 2026.

[47] MISRA, *MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems*, 3rd ed. Nuneaton, UK: MIRA Ltd., 2019.

[48] National Institute of Standards and Technology, *Post-Quantum Cryptography Standardization*. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography. Accessed: 2026.

[49] Arm Ltd., *TrustZone for Armv8-M*. [Online]. Available: https://developer.arm.com/Architectures/TrustZone-for-Armv8-M. Accessed: 2026.

[50] Texas Instruments, *EnergyTrace Technology*. [Online]. Available: https://www.ti.com/tool/ENERGYTRACE. Accessed: 2026.

[51] D. van Heesch, *Doxygen — Generate Documentation from Annotated C/C++ Sources*. [Online]. Available: https://www.doxygen.nl/. Accessed: 2026.

[52] LVGL, *Light and Versatile Graphics Library*. [Online]. Available: https://lvgl.io/. Accessed: 2026.

[53] Arm Ltd., *Mbed TLS — An Open-Source SSL Library*. [Online]. Available: https://www.trustedfirmware.org/projects/mbed-tls/. Accessed: 2026.

[54] Swedish Institute of Computer Science, *lwIP — A Lightweight TCP/IP Stack*. [Online]. Available: https://savannah.nongnu.org/projects/lwip/. Accessed: 2026.

[55] Texas Instruments, *Edge AI for Embedded Processors*. [Online]. Available: https://www.ti.com/technologies/edge-ai.html. Accessed: 2026.